Information Sciences 490 (2019) 344–368

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

Evolving fuzzy and neuro-fuzzy approaches in clustering, regression, identification, and classification: A Survey

Igor Škrjanc[a,*], Jose Antonio Iglesias[b], Araceli Sanchis[b], Daniel Leite[c], Edwin Lughofer[d], Fernando Gomide[e]

[a] Faculty of Electrical Engineering, University of Ljubljana, Slovenia
[b] Computer Science Department, Carlos III University of Madrid, Spain
[c] Department of Engineering, Federal University of Lavras, Brazil
[d] Department of Knowledge-Based Mathematical Systems, Johannes Kepler University Linz, Austria
[e] School of Electrical and Computer Engineering, University of Campinas, Brazil

ARTICLE INFO

Article history:
Received 27 November 2018
Revised 14 February 2019
Accepted 24 March 2019
Available online 28 March 2019

Keywords:
Evolving systems
Incremental learning
Adaptive systems
Data streams

ABSTRACT

Major assumptions in computational intelligence and machine learning consist of the availability of a historical dataset for model development, and that the resulting model will, to some extent, handle similar instances during its online operation. However, in many real-world applications, these assumptions may not hold as the amount of previously available data may be insufficient to represent the underlying system, and the environment and the system may change over time. As the amount of data increases, it is no longer feasible to process data efficiently using iterative algorithms, which typically require multiple passes over the same portions of data. Evolving modeling from data streams has emerged as a framework to address these issues properly by self-adaptation, single-pass learning steps and evolution as well as contraction of model components on demand and on the fly. This survey focuses on evolving fuzzy rule-based models and neuro-fuzzy networks for clustering, classification and regression and system identification in online, real-time environments where learning and model development should be performed incrementally.

© 2019 Published by Elsevier Inc.

1. Introduction

Progress of computer and communication technology has increased the capability to produce large amount of heterogeneous data from distinct autonomous sources endlessly. The amount of data increases continuously and changes rapidly over time. These data sets are called data streams. Data streams are common in online trading, financial analysis, e-commerce and business, smart home, health care, transportation systems, global supply logistic chains, smart grids, industrial control, cyber-security, and many other areas.

Data stream processing brings unique challenges which are not easily handled by many of the current computational intelligence and machine learning methods operating in batch off-line mode. This is because data streams are characterized by the following aspects [70]: i.) the sample samples in a stream arrive online, ii.) the system has no control over the order in which data samples arrive, iii.) data streams are typically unbounded in size as samples are sequentially recorded as long

* Corresponding author.
E-mail addresses: igor.skrjanc@fe.uni-lj.si (I. Škrjanc), jiglesia@inf.uc3m.es (J.A. Iglesias), masm@inf.uc3m.es (A. Sanchis), daniel.leite@deg.ufla.br (D. Leite), edwin.lughofer@jku.at (E. Lughofer), gomide@dca.fee.unicamp.br (F. Gomide).

https://doi.org/10.1016/j.ins.2019.03.060
0020-0255/© 2019 Published by Elsevier Inc.

as the whole system operates and iv.) once a sample is processed, it should be ideally discarded to keep virtual memory low and to cope with (constant) computational demands (ideally in real-time). Ideally, machine learning methods should readily adapt to changing situations occurring over the life-time of a stream. The data generation processes are emergent and dynamic over time, thus stream data processing methods must be capable to adapt to new situations (such as system drifts or non-stationary environments). An important question is how to transform stream data into knowledge. Machine learning and computational intelligence algorithms may fail when they encounter a situation that is distinct from the history embedded in historical data sets. Models are common in science and engineering, and in the development of meaningful models in the domain using data from non-stationary environments must allow models with the scope and granularity necessary to answer fundamental cause and effect relationships from new experiences. The demand of knowledge from data often entails interpretability of models in order to really understand the extracted knowledge. Rule-based models, whom evolving (neuro-)fuzzy systems belong to, typically offer better interpretable insights than pure neural networks or deep learning approaches.

Online learning is a powerful way to deal with stream data. An online learning algorithm observes a stream of examples to assemble a model and make predictions. It receives and uses immediate feedback about each prediction to improve model's performance. In contrast to machine learning and statistical learning schemes, online learning from data streams does not make assumptions on the distribution(s) of the observations. This is because the behavior it tries to predict changes over time in unforeseen ways, causing concept drifts and shifts. Concept drift denotes the way the data distribution changes gradually in time, whereas concept shift refers to a sudden, abrupt change of the characteristics of the data distribution. As data may evolve over time, data streams endow temporal locality. At the model level, the challenge is to develop global models by combining locally developed models to form a unified knowledge. This requires algorithms which are carefully designed in order to verify correlations among local models in the data-time space, and to combine the outputs from multiple local models in a proper way to achieve highly accurate overall models.

The impact of concept drift and shift in learning algorithms is enormous. While the effect of concept drift can be attenuated using e.g. model parameter adaptation procedures, concept shift may require a search in the underlying hypothesis space, which may be distinct from the current one. The key difference of evolving systems to online incremental machine learning (inc-ML) is their ability to simultaneously manage any significant changes (drift, shifts, non-stationary behaviors, environmental conditions etc.) in the system by using both, parameter *and* structural adaptation algorithms to process a data sample at most once (termed as *single-pass* property), while in inc-ML typically only parameters are updated, but no intrinsic structural change in the model is conducted.

Many types of data stream algorithms have been developed for clustering, classification, frequent pattern mining, anomaly detection, and numerous applications in distinct domains such as sensor networks, real-time finance, forecasting, control of unmanned vehicles, chaotic systems, and diagnosis have been reported [1,43,70,170]. Several algorithms and applications of evolving intelligent systems in clustering, classification, forecasting, control, diagnosis, and regression can be also found in the literature [18,132].

This paper gives a systematic survey on evolving systems, focusing on (neuro-)fuzzy systems in clustering, regression, identification, and classification. The purpose is to introduce the major ideas and concepts of evolving (neuro-)fuzzy systems, to provide an overview about architecture types and their main structural components as well as the respective basic incremental learning algorithms to address parameter adaptation and structural evolution properly. The paper also attempts to guide the reader to the essential literature, the main methodological frameworks and its foundations, and the design principles needed to develop applications as well as advanced concepts to make evolving (neuro)-fuzzy systems, E(N)FS, more robust and better applicable in real-world scenarios. The remainder of the paper is organized as follows. In the next section, the evolving systems are presented in general. An overview of various approaches comprising different evolving algorithms based on neuro-fuzzy models in clustering, regression, identification, and classification are given. In Section 3, the different mechanisms of adding clusters together with safety conditions and different ways of initializing new clusters, mechanisms for merging clusters, and mechanisms for splitting and removing clusters are discussed. Thereby, clusters are usually associated directly with components of (neuro-)fuzzy models, i.e. with rules and neurons. Section 4 discusses several important advanced concepts which were developed during recent years to improve robustness, convergence, generalization performance, usability and applicability of E(N)FS. At the end, some future directions and conclusion are given.

2. Evolving systems

Many systems are characterized by complex behaviors that emerge as a result of nonlinear spatio-temporal interactions among their components. Adaptation gives a system flexibility to improve its short-term performance, and increases its chances to survive in the long-term despite of changes in the environment and in its own components. While small changes in system parameters can be handled as a form of uncertainty, and can be properly addressed by using parameter estimation mechanisms, changes in the system structure requires a higher level of adaptation. An adaptive system is a nonlinear system that evaluates its performance, assesses the operating conditions of its components, measures the state of the environment, and adapts its dynamics to continuously meet performance specifications. In addition to parameter estimation, adaptation requires maintenance actions for performance and goal achievement (also termed as model maintenance) whenever large changes in the system structure and in the environment occur.

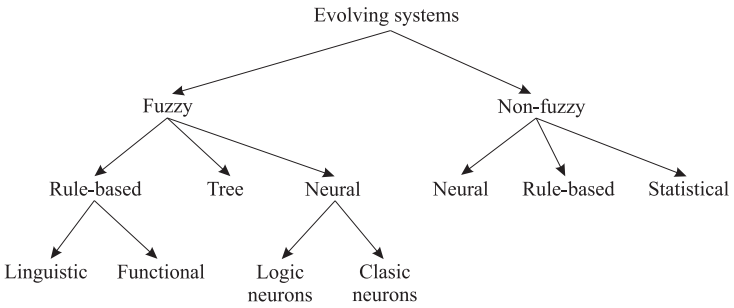


Fig. 1. Types of evolving systems.

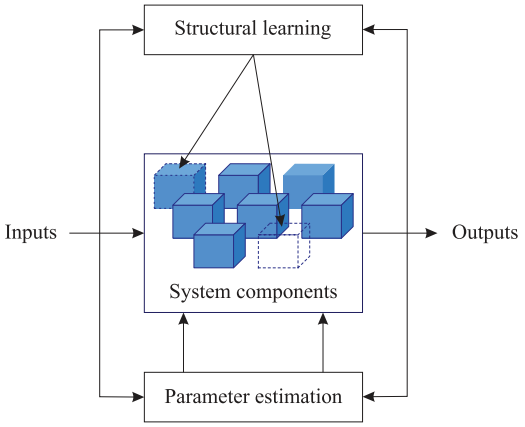


Fig. 2. Framework of evolving systems.

Adaptive and learning systems have been studied in science and engineering, especially in the area of adaptive control and system identification since the early fifties [35,188,189]. In adaptive control, the term adaptive means a class of design techniques which are applicable when the system model is partially known. These techniques either subsume some form of parameter adjustment algorithm [75], employ a set of finite local models and controllers with higher level supervisory switching [106], or use iterative learning techniques [3]. Adaptive control design techniques are mostly model-based, equipped with data-driven parameter estimation and self-tuning algorithms.

The field of *evolving systems* can be traced back to the year 1991 with the publication of the paper [149], where the method resource allocating network (RAN) was introduced. It deals with a neural network which can be adapted based on *gradient descent* learning and the *chain* rule to propagate errors backwards. Later the authors in [67] suggested a growing cell structure (GCS), which is a class of self-organizing neural networks that control structural changes using supervised or unsupervised learning. These papers did not attract much attention in the research community, perhaps because neural networks were not sufficiently established as a scientific discipline or stream mining topics were not really an issue at these days (the whole field of machine learning with the first journal arising in 1986 was in its infants, especially because of limited computer resources at this time). The field of evolving systems faced a tremendous development much latter (starting around the beginning of the 00s). Fig. 1 shows an overview of the different types of evolving intelligent systems.

Evolving systems are adaptive intelligent systems that, differently from adaptive and machine learning systems of the last decade, learn their structure and parameters simultaneously using a stream of data. The structural components of evolving systems can be artificial neurons, production rules, fuzzy rules, data clusters, or sub-trees [18,120]. The structure of rule-based systems is identified by the nature and the number of rules. For instance, evolving fuzzy rule-based systems may use linguistic fuzzy rules, functional fuzzy rules, or their combination. The structure of neuro-fuzzy systems is in turn recognized by the nature of the neurons, the network topology, and the number of neurons in the hidden layers.

Evolving intelligent systems as a framework to embody recursive data processing, single-pass incremental learning, and methods to develop systems with enduring learning and self-organization capabilities were first conceptualized in [13] when the term was coined. The authors use the term *evolving in the sense of gradual development of the system structure (rule-base or the architecture of the neural network that represents the system) and their parameters* as Fig. 2 shows. The authors also contrast the name *evolving* with *evolutionary* as used in genetic algorithms and genetic programming: while evolutionary processes proceed with populations of individuals using recombination and variation mechanisms during generations (typically in a temporally static, off-line optimization context), evolving processes advance *over time* during the life span of the system.

In summary, while adaptive systems in control and system theory deal predominantly with parameter estimation, and evolutionary algorithms with populations of models to produce new models, evolving systems benefit from learning from experience, inheritance, gradual change and knowledge generation from (temporal) streams of data [74].

Important milestones in the history of evolving systems can be mentioned such as the publication of the monographs: *Evolving Connectionist Systems* [99], *Evolving Intelligent Systems* [18], and *Evolving Fuzzy Systems — Methodologies, Advanced Concepts and Applications* [132,140]; and the beginning of the international journal entitled *Evolving Systems* [20] by Springer in 2010. In the next two subsections, we discuss the most important key approaches for evolving modeling which have appeared during the last 15 years, starting with clustering, regression and identification and then moving to classification methods.

2.1. Evolving systems in clustering, regression, and identification

This section overviews evolving algorithms for clustering, regression and identification. The emphasis lies on systems that we face in real life, namely systems that are nonlinear in nature and dependent on the influence of the environment, which may vary over time. This also means that the behavior of the systems changes over time. To deal with nonlinear and time-varying processes, the change of the behavior should be identified online, and ideally in real time. However, since the data are continuously generated from different sources, their amount is usually very large and samples can be highly heterogeneous and of very high dimension. Therefore, existing intelligent technologies should be adapted through the use of online learning algorithms so that big data streams can be processed in real time, [20,70]. The use of *off-line* methods in this kind of problem is not possible, [10], neither it is in the case of significant dynamic system changes and non-stationary environments (often appearing in complex real-world scenarios) [170]. This is especially important when the model of such systems is used in control, pattern recognition, monitoring or supervision.

In recent years, a number of successful evolving methods has been developed. The structure of the resulting models is usually based on fuzzy rules, neural networks or hybrid neuro-fuzzy concepts. Some important methods based on fuzzy models can be mentioned: eTS [10,14,24,161], xTS [29], + eTS [19], FLEXFIS [129], FLEXFIS+ [130], GS-EFS [139], IBeM [109,113], FBeM [112,115], and eFuMo [59].

Similarly, some of the most important neuro-fuzzy-based methods are: EFuNN [94,95], DENFIS [96], eGNN [114,116], GANFIS [31], SOFNN [121], SAFIS [163], SCFNN [126], NFCN [125], D-FNN [196], GD-FNN [197], SONFIN [91], NeuroFAST [191], RAN [149], ESOM [53], Neural Gas [68], ENFM [176], GAP-RBF [86], eFuMo [59], SOFMLS [165], PANFIS [152] and RIVMcSFNN [158]. The majority of the evolving methods used in regression is based on neuro-fuzzy local RBF models (*radial basis function models*) or on their generalized form, GRBF (GANFIS). The basic RBF models provide Gaussian membership functions with equal widths, as proposed in [197]. Other models suggest the use of ellipsoidal basis functions (EBF), which allow different widths of membership functions. This kind of approach is given in GD-FNN [197], and in SOFNN [121]. In eGNN, hyper-rectangles and trapezoidal membership functions with different widths are used. In [101] a new approach to evolving principal component clustering algorithms with a low run-time complexity for LRF data mapping is presented. In [181], a general evolving fuzzy model based on supervised hierarchical clustering for the design of experiments is described (see also Section 4.6). A general evolving fuzzy model for process control is shown in [202]. It is also remarkable that in SOFMLS and PANFIS an upper bound for the average of the identification error can be provided, which guarantees convergence and stability of the model (parameters), see also Section 4.5.

Evolving systems, similarly as adaptive neuro-fuzzy systems, learn from data streams by using learning algorithms to adapt their parameters in an online manner [194]. The parameters in this case are subdivided into linear and nonlinear ones. The nonlinear parameters, such as centers of clusters, width of radial basis functions, (inverse) covariance matrices or information granules, to mention a few, are related to the partition of the input-output space, whereas the linear parameters refer to the parameters of locally valid affine models. The partition of the input-output space is usually done by using different modifications (fuzzy) clustering methods, which are adapted for online usage from their off-line counterparts. These methods are unsupervised and aim at granulating the input-output space to achieve best possible representations of data streams (according to distance- and density-based criteria). The eTS method, for example, uses recursive clustering with subtraction [11] (*subtractive clustering* [47]). The ENFM method – a recursive version of the Gath-Geva clustering method – and eFuMo use recursive c-means and a recursive Gustafson-Kessel clustering algorithm [57]. To adapt local linear parameters, generally a recursive version of the least squares method, eventually with regularization, forgetting or weighting factor is employed. For example, FBeM [115] uses a specificity-weighted recursive least squares method.

Evolving fuzzy and neuro-fuzzy methods can also be divided according to the type of the model. Basically, the most frequent are the models that implement first-order Takagi-Sugeno fuzzy inference systems (SONFIN, D-FNN, GD-FNN, DENFIS, eTS, xTS, FLEXFIS(+), IBeM, FBeM, eGNN, NeuroFAST, SOFNN) or zero-order Takagi-Sugeno models (SCFNN, SAFIS, GAP-RBF, EFuNN). The essential difference between them is the use of a locally valid afine function or a constant in the consequent terms of the rules. Some evolving methods are based on generalized forms of fuzzy models, which consist of a combination of Mamdani, and first-order Takagi-Sugeno models (GANFIS, FBeM, eGNN, eMTSFIS [85]) and thus can achieve linguistic interpretation (due to the Mamdani part) with solid or high precision (due to the Takagi-Sugeno part).

Evolving methods can also be distinguished regarding the ability of adaptation. Notice that some fuzzy and neuro-fuzzy methods need the initial structure of the model (for example: GANFIS, ANFIS), which is obtained by *off-line* clustering. In this case, the number of fuzzy rules is constant during online operation and therefore the methods are not considered *evolv-*

ing methods, but *adaptive* methods since only parameter adaptation is performed online. The first methods to change the structure of the model were called incremental methods. These methods are equipped with mechanisms to add new local models or rules on demand, however they do not have mechanisms to delete old, useless or inactive rules. These methods include RAN, SONFIN, SCFNN, NeuroFAST, DENFIS, eTS, FLEXFIS. Some methods are also supplied with mechanisms to merge or combine clusters that are similar in some sense (ENFM, SOFNN). The incremental methods that are provided with procedures to delete and merge clusters are seen as real *evolving* methods. Some important fuzzy and neuro-fuzzy evolving methods are ESOM, SAFIS, SOFNN, GAP-RBF, Growing Neural Gas (GNG), EFuNN, IBeM, FBeM, eGNN, D-FNN, GD-FNN, ENFM, simpl_eTS, xTS, + eTS, FLEXFIS+, eFuMo, to mention a few. At this point, it is worth to mention alternative regression algorithms (which do not embed neuro-fuzzy components, but a different form of rule-based structure), in particular the incremental fuzzy linear regression tree algorithm of [119]. The algorithm starts with a single leaf with an affine model, and proceeds by replacing leaves with sub-trees. The algorithm uses a recursive statistical model selection test to update the tree.

2.2. Evolving systems in classification

This section overviews evolving algorithms in classification. Classification is the problem of identifying in which category a new observation belongs. In [51], the classification task is described formally as follows:

Given a set of training examples composed of pairs $\{x_i, y_i\}$, *find a function f(x) that maps each attribute vector* x_i *to its associated class* $y_i, i = 1, 2, \ldots, n$, *where n is the total number of training samples.*

An algorithm that performs classification is called a classifier. To train these classifiers, they receive as input a set of labeled data samples [84]. The training process can be carried out in off-line mode by considering all the data at once before the online operation of the classifier. In that case, it is assumed that a data set containing samples that represent all possible situations is available *a priori*. It is also assumed that changes of the trained classifier over time will not be required when new data arrive. This kind of classification approach is useful in some specific applications [34].

However, it is important to remark that, since the beginning of the 21st century, researchers have faced not only the problem of processing large data sets, but also handling data streams immediately after the samples arrive [56]. As mentioned before, since the data are continuously generated from different sources, they are usually very large in size and of very high-dimension. In addition, the data usually need to be processed in real time. Often, the training data set becomes available in small batches over time because the acquisition of these data continuously is expensive and time-consuming. For this reason, the development of classifiers which are able to manage continuous and high-volume data streams has taken place. Big, diverse and rapidly-produced data has also presented novel challenges in classification that are required to be tackled. These new data also provided opportunities to explore recently emerged scientific domains [73].

This new type of data and emerging needs are related to kinds of classifiers called incremental, which may update their parameters with new data samples. The development of incremental learning systems that can be trained over time from a data stream is a major open problem in the data mining area. An incremental classifier receives and integrates new examples without the need to perform a full re-learning phase from scratch. As discussed in a survey on supervised classification from data streams [117], a learning algorithm is incremental if for any example $x_1, \ldots, x_n$, it is able to generate hypotheses $f_1, \ldots, f_n$, such that f_{i+1} depends only on f_i and x_i, the current example. The notion of *current example* can be considered as the latest processed example. Incremental classifiers must learn from data much faster then the off-line mode classifiers. Thus, most of the incremental classifiers read the examples just once, process them through the update algorithm and discard them, afterwards — such a property is also called single-pass algorithm. In this way, they can efficiently process large amounts of data. Updates of model structures and parameters are mostly aimed to be carried out in a way such that convergence to the hypothetical batch solution (as achieved when using all the data samples at once for model training) is approached. This is an important criterion, because batch algorithms (mostly) produce optimal solutions in the sense of an underlying objective function.

Incremental classifiers have been implemented in many different frameworks:

- In relation to decision trees, the first incremental versions emerged in the 1980s. ID4 [117] and ID5R [192] concerned incremental classifiers based on ID3 (*Iterative Dichotomizer 3*) [162] – a well-known algorithm proposed by Quinlan in 1986. Later, in 2006, Ferrer-Troyano et al. [61] proposed a classification system based on decision rules that may store updated border examples to avoid unnecessary revisions when virtual drifts are presented in the data. Consistent rules classify new test examples by coverage, and inconsistent rules classify them by distance – similar to a nearest neighbor algorithm. The main characteristics of this approach is that the model is incrementally updated according to the new environmental conditions.
- Incremental classifiers have been implemented using neural networks [203]. An example of neural classifier is ARTMAP (*Adaptive Resonance Theory*) [40], a class of neural network architectures that performs incremental supervised learning in response to input vectors presented in arbitrary order. Later, a more general ARTMAP system [41] that learns to classify input data by a fuzzy set of features was introduced.
- In relation to a probabilistic framework, the Bayesian classifier is an effective methodology for solving classification problems when all features are considered simultaneously. However, sometimes, all the features do not contribute significantly to the classification. In addition, a huge computational effort is required when the features are added one by

one in a Bayesian classifier in batch mode using the forward selection method. For this reason, in [2], an incremental Bayesian classifier for multivariate normally-distributed data was proposed. In [46], several incremental versions of Bayesian classifiers are addressed.

- SVMs (*Support Vector Machines*) perform classification by constructing an n-dimensional hyperplane that optimally separates the data into two categories [193]. Support Vector Machine is a classical machine learning technique that can help multi-domain applications in big data environment [179]. However, SVM is mathematically complex, computationally expensive and furthermore requires a huge virtual memory in the case of larger data sets/streams, because of the kernel trick inducing a kernel matrix with a size of $N \times N$ with N the number of training samples. A training process on new data, discarding previous data, gives not optimal, but approximative results only. Considering this aspect, the authors in [44] propose an incremental procedure (an online recursive algorithm) for training SVMs using one vector at a time. In [198], an incremental algorithm that utilizes the properties of support vector sets and accumulates the distribution knowledge of the sample space through the adjustable parameters is proposed. The algorithm LASVM [39] is an online approach that incrementally selects a set of examples for SVM learning. A variety of incremental SVM algorithms is proposed in [55].
- In relation to lazy learning approaches, such as k-nearest neighbor (KNN), in [169] an incremental KNN algorithm is proposed. The algorithm is extended to a fuzzy version in [79]. These kinds of algorithms are useful when a variable number of neighbors are required for each point in the data set. However, lazy learning techniques are usually too slow to cope with (fast) online demands since a new model is built from scratch locally around each new query point (in dependency of the new query, in fact).

It is fundamental to remark that in these incremental methods the structure of the resulting classifier (a set of neurons, rules, clusters, support vectors, leaves, etc.) is fixed based on a prior choices. However, new data samples may not follow the same distribution of the training data, and it is necessary to face issues such as overfitting, low generalization and drift and shift of the density of the data stream [131].

Taking these considerations into account, the field of evolving intelligent classifiers started with the evolving fuzzy-rule based classifier eClass (*evolving Classifier*), [15,16]. An important aspect of eClass is that it can cope with large amounts of data and process streaming data in real time and in online mode. In addition, the different algorithms of the eClass family are single-pass, recursive, and therefore, computationally light since they have low memory requirements. eClass can evolve/develop from the new data; it has the following properties: eClass can start learning from scratch; and the number of fuzzy rules and classes do not need to be prespecified. These numbers vary by reading and analyzing the input data along the learning process. Thus, its structure is self-developed (evolved) over time.

In addition, eClass classifiers were categorized considering the consequent part of the fuzzy rules that cast the classifiers. In this sense, eClass includes different architectures and online learning methods. The family of alternative architectures includes: *eClass0*, with the classifier consequents representing class label (zero-order) and *eClass1*, which uses a first-order classifier. It is remarkable that recently, the zero-order classifier (*eClass0*) was demonstrated to be fully unsupervised [49].

eClass0 [16] is an FRB classifier and its structure follows the typical construct of an FRB classifier,

$$Rule^i: \textbf{ if } (x_1 \textit{ is around } x_i^1) \textbf{ and } \ldots \textbf{ and } (x_n \textit{ is around } x_n^i) \textbf{ then } L = (L_i) \tag{1}$$

where $Rule^i$ represents the ith fuzzy rule of the FRB structure, $x = [x_1, x_2, \ldots, x_n]^T$ is the vector of features, x^i denotes the prototype (existing sample) of the ith rule antecedent, and L_i is the label of the class of the ith prototype.

About the learning process of *eClass*, it is important to emphasize that FRB antecedent terms are formed from the data stream around highly descriptive prototypes in the input-output space per class. In the case of *eClass0*, its main difference to a conventional FRB classifier is that *eClass0* has an open structure and uses an online learning mechanism that considers such a flexible rule-base structure.

eClass1 [16] is an FRB classifier whose architecture regresses over the feature vector using first-order multiple-input multiple-output evolving Takagi-Sugeno (MIMO-eTS) fuzzy systems. The structure of an *eClass1* rule is

$$Rule^i: \textbf{ if } (x_1 \textit{ is around } x_i^1) \textbf{ and } \ldots \textbf{ and } (x_n \textit{ is around } x_n^i) \textbf{ then } (y^i = x^T\Theta), \tag{2}$$

where $Rule^i$ represents the ith fuzzy rule of the FRB structure, $x = [x_0, x_1, \ldots, x_n]^T$ denotes the $(n+1)$-dimensional vector of features, and y_i is the output.

A main aspect in the learning process of *eClass1* is the online identification of the parameters of the FRB structure. These parameters are updated with the arrival of new data samples carrying new information using recursive density concepts.

In [32], a new family of evolving classifiers is presented, namely simpl_eCLass0, which is an improvement of *eClass*. This family consists of two members: simpl_eCLass0 and simpl_eCLass1 (zero and first order classifiers). These classifier structures have all the advantages of the *eClass* family but their structure adjustment phase is simplified significantly, reducing computational overhead. In the same way as *eClass*, simpl_eCLass works in online mode updating the classifier/rules. In this case, the main differences of these two versions are the consequent part of the fuzzy rules, and their classification strategy, which is simplified based on the simpl_eTS+ approach [17].

A method for training single-model and multi-model fuzzy classifiers incrementally and adaptively is proposed in [128]. This method is called FLEXFIS-Class as its core learning engine is based on several functionalities as described in the original FLEXFIS approach [129] (including the rule evolution concept). In [128], two variants of evolving fuzzy classification schemes are presented:

- FLEXFIS-Class SM is an evolving scheme for the single-model case. It exploits a conventional zero-order fuzzy classification model architecture with Gaussian fuzzy sets in the antecedent terms, crisp class labels in the rule consequents and (fuzzy) confidence values for each class in each rule.
- FLEXFIS-Class MM is based on a multi-model architecture that exploits the idea of nonlinear regression by an indicator matrix to evolve a Takagi-Sugeno fuzzy model for each separate class (receiving a label of 1 while all other classes receive a label of 0). To provide a final classification statement, the maximal output value from all fuzzy models is elicited: the final class output corresponds to the maximum argument, i.e. the output is the class represented by that model that produced the maximal output value.

In [136], the authors extended FLEXFIS-Class to another multi-model variant in the case of multi-class classification problems by using the all-pairs technique, then termed as EFC-AP (Evolving fuzzy classifiers with All-Pairs). For each class pair either a binary FLEXFIS-Class SM model (EFC-AP SC) or a Takagi-Sugeno fuzzy model (by regression on {0, 1}, EFC-AP TS) is established. For a new query point, a preference value for each class-pair is elicited (how much one class is preferred over the other according to the output confidence), which can be stored in a preference relation matrix. This matrix can be analyzed to produce a final classification statement. Due to the all-pairs technique, the problem of class imbalance in stream learning (leading to deterioration in performance on under-represented classes) could be reduced. This was successfully evaluated when introducing new classes on the fly in a streaming context for online visual inspection systems [137]. A significant increase in classification accuracy on new classes (under-represented after their birth) could be observed when using EFC-AP, compared to FLEXFIS-Class SM/MM.

In [21], a new method for defining the antecedent part of a fuzzy rule-based classification system, called AnYa, is proposed. The method removes the need to define the membership functions per variable using often artificial parametric functions such as triangular, Gaussian etc. Instead, it strictly follows the real data distribution by using the concept of data clouds, which can be applied to classification tasks. In addition, as it is based on vector forms, logical connectives are useless. Finally, it uses *relative data density* expressed in a form of a parameter-free (Cauchy type) kernel [22] to derive the activation level of each rule. These levels are then fuzzily weighted to produce the overall output. In this case, AnYa-Class uses a single rule for each class since all the data of a class form a single data cloud. The number of rules is fixed, so this classifier is incremental, but not (fully) evolving. AnYa-Class, as the *eClass* family, is divided in two types: zero order if the consequent of each rule is a single class label, and first order if the consequents of the rules are linear. The concept was used in control, [26], to construct the Robust evolving cloud-based controller (Recco) [7] for heat-exchanger plant, and in [8] for real two-tank plant control. This kind of structure was also used in model identification of production control [6] and for evolving model identification for process monitoring and prediction of nonlinear systems in general [9]. Monitoring of large-scale cyber attacks using evolving Cauchy possibilistic clustering is shown in [185]. A successful implementation is also reported for evolving cloud-based system for the recognition of drivers' actions in [184]. A comparison of approaches for identification of many data-cloud-based evolving systems is presented in [37]. The problems of identification of cloud-based fuzzy evolving systems are studied and elaborated in [38] and a robust fuzzy adaptive law for evolving control systems is presented in [36,183]. In [186] the unification and generalization of different evolving clustering methods based on Cauchy density is presented. Different inner matrix norms are implemented for clustering, classification and regression from data streams.

A different version of the *eClass* family, called AutoClassify, is proposed in [23]. As *eClass*, the *AutoClass* family works on a per-sample basis, and requires only the features of that sample plus a small amount of recursively updated information related to the density. In addition, depending on the form of the consequent part of the rules, *AutoClassify* includes:

- AutoClassify0, which is a fully unsupervised method. The learning phase of *AutoClassify0* is unsupervised and based on focal points by clustering or partitioning in data clouds. The term data clouds is proposed in AnYa [21] and refers to structures with no defined boundaries and shapes.
- AutoClassify1 generally provides a better performance compared to its counterpart, but it is semi-supervised and takes advantage of more parameters. AutoClassify1 can work as a MIMO type model for multi-class classification problems. The learning phase of this classifier is based on the decomposition of the identification problem into overall system structure design and parameter identification. However, these tasks are performed in online mode, sample per sample.

A systematic framework for data analytics is proposed in [93]. The underlying classifier is based on the typicality and eccentricity of the data, and is called TEDAClass (*Typically and Eccentricity based Data Analytics Classifier*). This classifier is evolving, fully recursive, spatially-aware, non-frequentist and non-parametric. TEDAClass is based on the TEDA method [25]. It uses local typicality and eccentricity to calculate the closeness between data and fuzzy rules.

In [164], an Extended Sequential Adaptive Fuzzy Inference System for Classification, called ESAFIS, is presented. It is based on the original SAFIS approach [163], which itself is based on the functional equivalence between a radial basis function network and a fuzzy inference system. The SAFIS algorithm consists of two aspects: determination of the fuzzy rules and adjustment of the premise and consequent parameters in fuzzy rules. ESAFIS extends SAFIS to classification problems and proposes some modifications in calculating the influence of a fuzzy rule, adding fuzzy rules and especially a faster RLSE based estimation of consequent parameters to speed up the learning process. In [168], a new algorithm is proposed as the combination of SAFIS and the stable gradient descent algorithm (SGD) [167]. The modified sequential adaptive fuzzy inference system (MSAFIS) is the SAFIS with the difference that the SGD is used instead of the Kalman filter for updating the parameters.

Evolving semi-supervised classification is discussed in [108]. The granulation method used to construct the antecedent part of evolving granular predictors, often referred to as eGM (*evolving Granulation Method*), is applicable to the partition of unbalanced numerical and granular-valued partially-supervised streaming data subject to gradual and abrupt changes. If an unlabeled sample causes the creation of a granule, then the class of the granule remains undefined until a new labeled sample falls within its bounds. The class label of the new sample tags the granule. Contrarily, if an unlabeled sample rests within the bounds of an existing granule whose label is known, it borrows the granule label. Core and support parameters of trapezoidal fuzzy sets are adapted to represent the essence of the data stream. More abstract, high-level granules are easier to manage and interpret.

Ensemble learning has also been used in evolving frameworks. Ensemble learning is a machine learning paradigm in which multiple learners are trained to solve the same problem [151] and where the diversity of so-called weak learners (e.g., simple fuzzy classifiers with low number of rules) can improve the prediction accuracy when being combined [150] — Learn++ was one of the first method to address ensemble learning in an incremental context, but it is not evolving. In this sense, [88] presents a method for constructing ensembles based on individual evolving classifiers. In [89], a scheme for constructing ensembles which are created considering the idea behind the stacking technique [195] is addressed. In addition, an evolving ensemble classifier, termed parsimonious ensemble (pENsemble) is proposed in [159], where local experts (base classifiers) are weighted according to their classification accuracy: models with low weights are discarded to make the ensemble more compact. Base classifiers are added on the fly whenever a drift is confirmed by a drift detector based on Hoeffding's inequality. The base classifiers themselves are internally updated and evolved with the use of the pClass method [154]. Recently, it has been successfully applied in an extended variant for online tool condition monitoring in [160]. TEDA, eTS and xTS are combined as an ensemble in [175], where diversity among their outputs is exploited in order to improve classification accuracy.

Since clustering can be defined as an unsupervised classification of observations into groups (clusters) according to their similarity, it can be considered as a type of classification. This well-known unsupervised classification problem has been solved by a variety of off-line approaches such as k-Nearest Neighbor, fuzzy c-means or density-based methods. The recursive version of the latter was first reported in [58]. A Gustafson-Kessel modification to allow ellipsoidal forms of clusters is given in [57]. Other well-known incremental/online approaches are the Self-Organizing Maps, SOM [103] – which was extended in [54] to an evolving approach – and Adaptive Resonance Theory, ART [42]. However, these approaches are not fully unsupervised and autonomous since some problem-specific thresholds and guesses on the number of clusters in the data set are required. In this respect, evolving methods are different since they can start learning from scratch with no need of initial information. Moreover, the number of clusters depends on the data and should be automatically evolved on demand and on the fly.

Considering these aspects, the notion of autonomous clustering was pioneered with *eClustering* [12], an evolving clustering approach based on the potential/density of the data samples, which is recursively calculated by using RDE [16]. In such clustering method, the first data sample represents the first cluster center. The density of the subsequent data samples is calculated using RDE when they arrive. A new data sample represents a new cluster center if it has higher descriptive power than any of the other centers. In addition, the algorithm checks if the existing clusters should be removed or cluster parameters should be adapted. Similar to *eClass, eClustering* is one pass, non-iterative, recursive and can be used interactively. In [17], an improvement of *eClustering*, called *eClustering+*, which does not rely on user- or problem-specific thresholds is proposed. It estimates the density at a data point using a Cauchy function.

In [97], an evolving clustering method (ECM) that employs a type of fuzzy inference, denoted as *dynamic evolving neural-fuzzy inference system* (DENFIS) is proposed. ECM does not ask for the number of clusters, and cluster centers are represented by evolved nodes. In this case, a threshold value to define the maximum distance between a data sample and cluster centers is required.

An evolving version of the Gustafson-Kessel (GK) algorithm [76], called eGKL (evolving Gustafson-Kessel-like), is proposed in [63]. eGKL provides a methodology for adaptive, step-by-step identification of clusters that are similar to GK clusters. In this sense, eGKL estimates the number of clusters and recursively updates its parameters based on the data stream. The algorithm is applicable to a wide range of practical time-varying issues such as real-time classification. In [182], the idea of evolving Gustafson-Kessel possibilistic c-means clustering (eGKPCM), as an extension of the PCM clustering algorithm, is introduced. PCM is given in [105].

In [33], an online evolving clustering approach from streaming data that extends the mean-shift clustering algorithm is proposed. The algorithm is called Evolving Local Mean (ELM), because it uses the concept of non-parametric gradient estimate of a density function using local mean. An ELM prototype consists of a cluster center and a distance parameter. The approach is defined as evolving since the local mean is updated from the data stream and new clusters are added to its structure when the density pattern changes.

Finally, autonomous split-and-merge techniques for assuring homogeneous and compact prototype-based clusters in an incremental, single-pass learning context are proposed in [135,138]. These are based on conventional and extended evolving vector quantization (EVQ) concepts – the latter leading to arbitrarily rotated and shaped clusters with the use of a recursive estimation of local inverse covariance matrices. Cluster evolution is decided based on a statistical tolerance region using the prediction interval.

In general, the cluster identification algorithms and approaches mentioned in this section should be flexible to changes in the data stream. The number of clusters, cluster sizes and associated parameters should ideally be significantly chang-

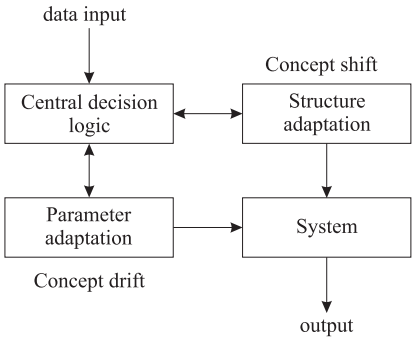


Fig. 3. Basic evolving method.

ing whenever demanded, while still assuring approximative convergence of the model to the (hypothetical) batch solution (achieved when seeing all data samples at once), especially during regular modes in the stream process. Therefore, the stability-plasticity dilemma [145] should be somehow appropriately addressed within any incremental learning method and evolving framework. We address this issue also in more detail in Section 4.5 The next section discusses the main differences of evolving algorithms according to the mechanisms of adding, deleting, merging, and splitting local models.

Next, we study different evolving concepts and mechanisms which are necessary to cope with the dynamically changing aspects of data streams properly, including adding, removing, merging and splitting of clusters and model components, as well as initial and safety conditions.

3. Different evolving mechanisms

Evolving systems should change the structure of the model that describes the behavior of the data stream and be able to adapt parameters associated to local models. The latter is generally dealt with by using some version of recursive or weighted recursive least squares. The most challenging task, and also the basic feature of the evolving systems, is therefore related to adding, deleting, splitting and merging of clusters, neurons, granules or clouds, in order to assure significant flexibility in the case of changing system situations and non-stationary environments during the incremental stream learning process.

Basic constituting elements of evolving intelligent systems can be defined. Fundamentally, these systems consist of three basic blocks, as shown in Fig. 3. The main block is the *central decision logic* block. This block calls the remaining, *adaptation* and *evolving*, blocks whenever necessary. In the adaptation block, the local model, rules or cluster parameters are adapted according to incoming data samples that belong to the region of the data space already covered by the local model. By contrary, in the evolution block, the structure of the global model is changed (e.g., it is expanded when samples are not covered by the current model and thus embed significant novelty content). In other words, parameter adaptation is useful to model gradual or slight changes of behavior (*concept drift*) or simply to reduce model/parameter uncertainty due to previous data insignificance (also termed as *model refinement*), while structural evolution is useful to fit new patterns or completely different behaviors or events into the model (*concept shift*).

The basic ideas behind evolution mechanisms are very different and suitable for different tasks. Next, these mechanisms and corresponding algorithms are discussed in more detail.

3.1. Adding clusters (and neurons)

Cluster adding is the most essential mechanism of evolving systems. Usually, learning starts with no local models or clusters; they are added to the global model on the fly in order to expand its knowledge to new regions of interest in the feature space (reducing extrapolation likelihood for new query points). Fig. 4 shows a typical two-dimensional example (with features X_1 and X_2) of cluster adding versus cluster adaptation in the case when new data samples are coming in (rectangular markers): i.) when the new samples contain significant novelty content (e.g., lying far away from existing clusters when using a distance criterion), a new cluster is added to best represent the new samples; ii.) when the new samples are close or falling into an already existing cluster, this cluster is updated (its ellipsoidal region expanded in our example, which could be achieved by updating the inverse covariance matrix of the cluster, for instance).

After adding a cluster, a very important task concerns the *initialization* of the parameters of the new local model. Another key decision is related to when and in which place of the data space to consider the cluster. Such decision usually depends on *thresholds*. These thresholds can be given according to (i) the output error – the error between the current measured output and the estimated model output; (ii) some distance, similarity or density metric regarding the current measured input data and cluster prototypes (centers generally); or (iii) the condition of ϵ-completeness, which is connected to the membership degree of the current sample in the current clusters. In the case of fuzzy systems and other types of

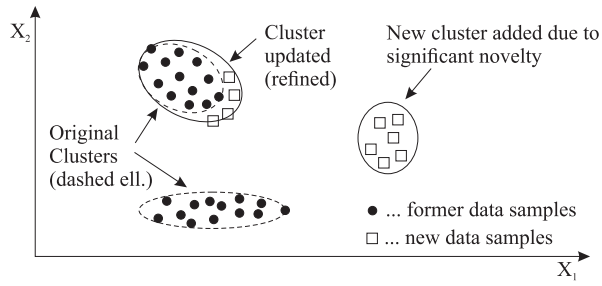


Fig. 4. Example of cluster adding versus cluster update based on new incoming data samples (rectangular markers).

architectures embedding local granular descriptors of partial regions with a geometric meaning, clusters are usually directly associated with model components (e.g., with fuzzy rules). Thus, cluster adding goes hand in hand with component adding.

The criteria to add a new neuron in the case of evolving systems which are based on neural networks are quite different. In the case of GNG, [69], a new neuron is added after n new samples have been loaded, where n stands for a user-defined constant. In many cases, a criterion is defined according to the Euclidean distance between the current sample and cluster centers. This criterion is used in the case of ESOM, [53], DENFIS and FLEXFIS. This means that such a criterion is used in an unsupervised manner and thus no target value (e.g., a label) needed. For supervised learning, adding criteria are generally based on the error between the measured and estimated outputs together with some logic and conditions regarding the distance to the cluster centers. This is taken into account in the following methods: EFuNN, D-FNN, GD-FNN, SAFIS [163], SCFNN. The condition for cluster addition can also be given in the form of ϵ*-completeness*, which is used in RAN, SCFNN, SONFIN, eTS. This condition defines the minimal allowed membership value of a new sample to the current rules.

In [80], DFKNN considers an adding mechanism based on the Euclidean distance from a sample to the cluster centers and on the change of the local variance caused by the sample. To add a new cluster, the distance and the variance should be greater than a given threshold. As additional condition, the number of samples that belongs to a cluster, is monitored. If this number is greater than a threshold, defined by the user, then a new cluster is created.

In [50], a dynamic data clustering algorithm is presented. Cluster addition takes into account the distance between the current sample and the cluster centers, which should be larger than half of the minimal distance between two cluster centers. Moreover, the membership degree of the sample in the clusters should be greater than a pre-defined threshold.

In the case of DENFIS [96], cluster addition is based on a generalized Euclidian distance. If the current sample is within the radius of at least one of the clusters, then the model is not changed. Contrarily, the sum of the cluster radii and the distance between the current sample and the center of the chosen cluster is calculated. This is done for all clusters. If the minimal sum is larger than the double of a threshold value, then a new cluster is added; otherwise, the parameters of the cluster are adapted. The threshold is equal to the maximum allowed cluster radius.

The algorithms D-FNN [196] and GD-FNN [197] are similar. Cluster addition is performed according to the output error and the distance of the current sample to the current centers of the clusters. If only the output error is greater than a threshold, the parameters of the local models are adapted. If only the distance is larger than the threshold, the parameters of membership functions are adapted. Otherwise, if both are above the threshold, than a new cluster is added. The thresholds are adaptive. At the beginning, they assume higher values, which are reduced over the iterations. This means that initially a general model is obtained, which becomes more detailed over time. This is accomplished by decreasing the thresholds. The difference between D-FNN and GD-FNN is the way the thresholds are adapted. The method RAN [149] is also similar, but it uses constant thresholds.

The NeuralGas algorithm [68] monitors the accumulated error between the measured output and the output of the system model in a prescribed time interval. If the error exceeds a predefined threshold, then a new cluster is added. A very similar approach is performed by the NeuroFAST algorithm [191]. The algorithms GAP-RBF [86] and SAFIS [163] add new clusters according to the output error and the distance to the active cluster. In the meantime, the improvement of the model error is calculated in the case a new cluster would be added to the position of the current measured sample. If these three criteria are fulfilled, namely, the output error is larger than a threshold, the minimum distance to the cluster centers is larger than a threshold, and sufficient model improvement in relation to the reduction of the output error is observed, then a new cluster is added.

The criterion for cluster addition in the case of EFuNN [94,95] is based on sensitivity, which is a function of normalized distances. The NFCN [125], ENFM [176], SONFIN [91], SCFNN [126] and SOFNN [121] algorithms are based on the principle of ϵ-completeness, which means that the minimal membership degree of the current sample to all clusters (rules) should not be smaller than a predefined threshold. The SOFNN and SCFNN algorithms take into account not only the ϵ-completeness criterion, but also an additional criterion based on the variation of the output error.

In the case of eTS [10,161], cluster addition is based on the potential of a current sample. The sample is accepted as the center of a new cluster if the distance to the closest center exceeds a predefined threshold and the potential of the sample is larger than the potential of the current clusters. If the distance condition is not fulfilled, the closest cluster center moves

toward the sample. Otherwise, if the distance condition is fulfilled, but the potential of the candidate is lower than the potential of the centers, then only the parameters of the local models are adapted.

Granular evolving methods, IBeM [109,113], FBeM [112,115] and eGNN [114,116], consider a maximum expansion region (a hyper-rectangle) around information granules. Granules and expansion regions are time-varying. They may contract and enlarge independently for different attributes based on the data stream, the frequency of activation of granules and rules, and on the size of the rule base (IBeM, FBeM) or neuro-fuzzy network (eGNN). If a sample does not belong to the expansion region of the current granules, a new granule is created. In eGNN, the use of nullnorm and uninorm-based fuzzy aggregation neurons may provide granules with different geometries [114].

FLEXFIS [129] and its classification versions FLEXFIS-Class SM and MM [128] add a new cluster according to the distance between a sample and the cluster centers. The cluster is added if the smallest distance exceeds a *vigilance parameter* which is normalized subject to the current input dimension in order to avoid too intense cluster growing due to curse of dimensionality. GS-EFS [139] adds a new cluster (in arbitrarily rotated position) according to the Mahalanobis distance between a sample and its nearest cluster. The statistical estimation of the so-called *prediction interval* by using an approximated, fast version of the Ξ^2-quantile serves as tolerance region around the ellipsoidal cluster contour in order to decide whether a new (generalized) rule should be evolved or not.

In the eFuMo algorithm [59], the decision about adding clusters can be based on the Euclidean or Mahalanobis distance regarding the current sample and the cluster centers. Calculations can be based on all or on particular elements of the data and cluster vectors.

IN PANFIS [152], a new cluster is added whenever the model error on the new sample is high and also its significance to the PANFIS overall output is given (both factors are multiplied). The latter is measured by the integration of the winning rule (nearest one to the current samples) over the complete feature space, normalized by its range: in order to avoid the revisit of past samples, this can be approximated with determinant operations on covariance matrices representing the shapes and orientations of generalized rules.

It is recommended to consider multiple criteria and different conditions for cluster addition. This is performed by NEUROFast and eFuMo. In eFuMo, the concept of *delay of evolving mechanisms* is introduced. The delay is an interval in which evolving mechanisms are not enabled. Only adaptation of centers and model parameters is conducted during such time interval. The delay of evolving mechanisms takes place after a change in the structure of the system performed by any evolving mechanism. The model should have a certain period to adapt on the new structure. The duration of the delay should be defined by the user and depends on the data. Additional safety conditions are discussed next.

3.1.1. Safety conditions

When evolving algorithms are based on Euclidean distance there may be regions inside a hypershpere with no representative data. This is generally not true if the Mahalanobis distance is used because the distances in this case are normalized by the variance of the attributes. This allows multiple ellipsoids to develop close to each other but oriented with different angles.

In real-world data streams, some issues may arise when evolving models deal with outliers. Ideally, outliers should not cause the creation of a new cluster. Therefore, an additional safety condition is generally given, and should be tested before adding clusters to a model. In eFuMo, this condition is based on the number of output samples that do not belong to the current clusters. A delay is introduced into the adding mechanism, but the addition of unnecessary clusters is prevented. The safety condition is given as: *a new cluster is added if N consecutive output samples belong to the same cluster and fulfill the necessary criteria for cluster addition.* The probability of adding a cluster as a result of outliers is decreased to $P(x)^N$, where $P(x)$ stands for the probability of forming a new cluster from an outlier. The number of samples N is usually chosen from 5 to 10. Similarly, FLEXFIS(-Class) and GS-EFS embed a rule-base procrastination option, where, after adding new clusters, several samples are awaited before the cluster becomes significant and thus is considered as a new rule in the rule base.

Some evolving methods accept the creation of clusters in a passive way. In this case, the cluster added to the model based on an outlier will probably not be activated for a number of iterations. Deleting procedures play a key role in these methods to keep the rule base concise and updated.

Next, we discuss the initialization strategies in the case when a cluster/component is added.

3.1.2. Initialization of a new cluster

When a sample fulfills all conditions for adding a cluster/rule/component, it usually defines the new cluster center. A second parameter to be defined is the size of the cluster. In ellipsoid-based models, the size depends on the covariance matrix. In the literature, a number of different initialization approaches is given: the size of the new cluster depends on the distance to the closest cluster (DENFIS [96], D-FNN [196]); the initial covariance matrix is fixed and given as a user defined parameter (SCFNN [126], SONFIN [91]); it can also be given as the average of the covariance matrices of the existing clusters (xTS [29]). In ENFM [176], the covariance matrix is equal to the covariance matrix of the closest cluster, and in FLEXFIS [129] it is set to a small value of ϵ to guarantee numerical stability of the rules and fuzzy sets. In GS-EFS [139], the inverse covariance matrix is initialized as a fraction of the range of the input features or by a weighted average of neighboring rules (where the weights are the support of the rules, i.e. number of data samples that formed them). In PANFIS [152], the covariance matrix is initialized as a diagonal matrix in a way that ϵ-completeness is guaranteed (similarly as in SONFIN), i.e.

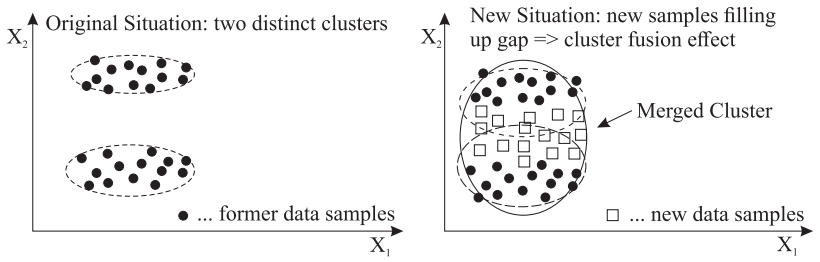


Fig. 5. Example of cluster fusion effect (right plot) in the case of data samples filling up the hole between originally distinct clusters (as shown in the left plot); the merged cluster is shown as solid ellipsoid in the right coordinate system.

achieving a minimal overlap degree of ϵ with any of the adjacent clusters. Initialization based on the distance to the closest cluster is successful because it covers the gap between clusters. Gap covering was discussed, for example, in [107].

Parameters of local linear models should also be initialized. In [10], the parameters are initialized as 'zero' in the case of using the local fuzzy least squares algorithm (for estimating the consequent parameters in each rule separately), and as the weighted average of the other local linear models in the case of using the global least squares algorithm (for estimating the consequent parameters in all rules together). The weights are the membership values. In [96], the parameters of the new local model are equal to those of the closest local model.

Initialization of local linear model parameters by a weighted average [10] is common. Together with the initialization of local model parameters, covariance matrices can also be taken into account. Weights used to initialize the new local model parameters may consider the variance of a certain parameter. When a new local model is added, covariance matrices in recursive algorithms can be multiplied by a factor $\rho = \frac{c^2+1}{c^2}$, where c is the number of current clusters. This makes further adaptations more stable.

The next two subsections deal with two important extensions, namely *cluster merging* and *cluster splitting* in order to assure homogenous clusters and compact model components and structures.

3.2. Merging clusters

Merging is necessary if clusters significantly overlap with each other due to adaptation mechanisms. This effect is called cluster fusion and is usually caused by consecutive samples belonging to the gap in-between two or more clusters, which is used to be disjoint at a former time instant, but latter it turns out that they are not and should be merged to eliminate redundancy. A two-dimensional visualization example (with features X_1 and X_2) of this effect is shown Fig. 5.

Merging of clusters does not only provide a more accurate representation of the local data distributions, but also keeps E(N)FS, in general, more compact and thus more interpretable and adaptable.

Different mechanisms for cluster merging are given in this section. In DKFNN, the algorithm monitors the positions of the clusters centers. If two of them approach one another, the underlying clusters should be merged. A measure of cluster similarity, useful for merging, is given in [66]. The measure is based on the membership degree of samples in clusters and is similar to the correlation between the past activations. Merging based on correlation among previous activations is also given in [95] (EFuNN). In this algorithm, merging is based on the maximum cluster radius. Neighbouring clusters that present the sum of radii less than a maximum threshold are merged. In ENFM, two clusters are merged when the membership of the first cluster center to the second cluster is greater than a predefined threshold, and vice-versa. In SOFNN [121], clusters are merged when they exhibit the same centers, which is almost impossible in practice.

The algorithm FLEXFIS+ [130] calculates the intersection of the membership functions in each dimension. This is the basis to define the index of overlapping, which is then used to judge whether whole clusters (rules) should either be merged or not. If the index is greater than a predefined threshold, then clusters are merged. Merging itself is conducted in the antecedents by an extended variant of recursive variance formula and in the consequents by exploiting Yager's participatory learning concept [199] in order to resolve possibly conflicting rules properly. GS-EFS adds a homogeneity condition among both, antecedent and consequent spaces, to decide whether two clusters should be merged: the angles between their hyper-planes should not be too small and their joint volume should not be too large. This assures that clusters are not merged inappropriately when they are actually needed to handle local nonlinearities.

The eFuMo algorithm merges clusters based on the normalized distance between their centers. The distance is calculated based on the Mahalanobis measure. The parameters of the merged cluster are initialized by a weighted average [176] or using a normal average, such as in [95], while the merged covariance matrix can be defined as proposed in [123]. The algorithm uses also the parameters of the local model similar to FLEXFIS+, but eFuMo also takes into account the prediction of the local models. Three conditions for merging are: angle condition, correlation condition, and distance ratio condition. Two clusters are merged if they fulfill one of these conditions. Additionally, a condition for the local model outputs is taken into consideration. The difference between two outputs should be less than a predefined threshold, and they should have a support set higher than a predefined value.

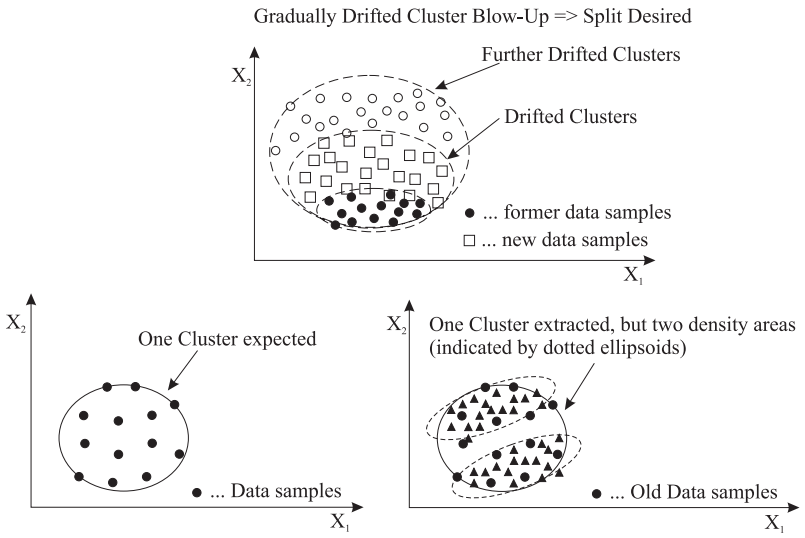


Fig. 6. Upper: cluster blow-up due to gradual drifts always updating the cluster a bit (no cluster evolution triggered); middle: showing a cluster delamination effect due to samples originally appearing as one cluster; lower: but, when new samples fall into this cluster, two heterogenous data clouds turn out.

An instantaneous similarity measure is introduced in FBeM [112] for multidimensional trapezoidal fuzzy sets, say A^{i_1} and A^{i_2}, as

$$S(A^{i_1}, A^{i_2}) = 1 - \frac{1}{4n}\sum_{j=1}^{n}(|l_j^{i_1} - l_j^{i_2}| + |\lambda_j^{i_1} - \lambda_j^{i_2}| + |\Lambda_j^{i_1} - \Lambda_j^{i_2}| + |L_j^{i_1} - L_j^{i_2}|), \tag{3}$$

where $A^i = (l^i, \lambda^i, \Lambda^i, L^i)$ is an n-dimensional trapezoid. Such measure is more discriminative than, for example, distance between centers of neighbouring clusters, and its calculation is fast. If $S(A^{i_1}, A^{i_2})$ is less than a maximum width allowed for clusters, the underlying clusters are merged. The cluster that results from the merging operation takes into account the bounds (convex hull) of the combined clusters to provide the highest level of data coverage.

3.3. Splitting clusters

The splitting of clusters is defined for a finer structuring of the data space and the model structure. Basically, an evolving algorithm should, in the case of regression and identification problems, accept a larger number of clusters in the region where the model output error (approximation or prediction error) is greater than the expected one or grows extraordinary. This can happen because clusters may grow over time due to gradual drifts, cluster delamination effects or inappropriately-set (pessimistic) cluster/rule evolution thresholds. Fig. 6 shows an example for gradual drifting patterns and delamination effect, leading to blown-up rules. The latter is the case when evolving methods are used in new applications in a plug-and-play manner by using parameters tuned to past tasks or problems.

The concept of splitting is proposed in [78] and in [52]. A Chernoff measure is used in the former while a *fidelity measure* is assumed in the latter. The author in [135] proposes a penalized BIC (*Bayesian information criterion*) to decide whether the current cluster structure should be kept or whether the latest updated cluster should be split into two (the partition receiving a lower penalized BIC value is preferred). The penalization of the log-likelihood is extended with a product term, which vanishes in case of close over-lapping clusters, thus punishing them more than clearly disjoint clusters. As the punished BIC may not fully represent real cluster homogeneity versus cluster heterogeneity, the split approach in [138] (AutoClust) extends this approach by applying a Gaussian mixture model estimation along each principal component direction of an updated cluster with two Gaussians and then checking whether any two Gaussians (in each direction) are significantly different (according to the Welch test). If so, a heterogeneous cluster is found (i.e. a cluster that represents two disjoint data clouds internally), and thus it should be split. The split point is estimated through the cutting point of the adjacent (but statistically different) Gaussians.

In NeuroFAST [191], clusters are split according to their mean square error (MSE). The algorithm calculates the error in each P steps and splits the cluster and the local model with the greatest error. The mechanism of splitting in eFuMo is based on the relative estimation error, which is accumulated in a certain time interval. The error is calculated for each sample that falls in one of the existing clusters. The initialization of the resulting clusters is based on the eigenvectors of the cluster covariance matrix, as in [81] and in [187] where nonlinear system identification by evolving Gustafson-Kessel fuzzy clustering and supervised local model network learning for a drug absorption spectra process is presented.

An innovative and efficient (fast) incremental rule splitting procedure in the context of generalized evolving fuzzy systems (extending GS-EFS approach [139]) is presented in [143] for the purpose of splitting blown-up rules with high local estimation errors over the past iterations. In this sense, it can autonomously compensate those drifts, which can not be automatically detected (e.g., slowly gradual ones), see Section 4.2.

Finally, in the next subsection, we discuss how the removal of superfluous clusters/component, which typically became superfluous over time, but earlier were important, is addressed in the current evolving modeling approaches.

3.4. Removing clusters

Mechanisms for removing clusters are convenient to delete old or inactive local models, which are no longer valid. These mechanisms are of utmost importance in classification and pattern recognition to assure faster computation speed in adaptation and more compact rule bases and networks. In general, it happens that a cluster is created in a part of the input-output space where there are just a few representative samples. This is justified by errors in measurements or due to a change of the system behavior so that a cluster is not useful after a number of iterations. These clusters can be removed from the model, because they do not help in the description of the data. Nonetheless, care should be taken with seasonal behaviors since a cluster may be reactivated latter. Moreover, in anomaly detection problems, unusual and idle clusters may be more important than those highly operative clusters, and therefore should not be removed.

The mechanisms to remove clusters are mainly based on the following principles: the age of the rule (xTS, GNG, ESOM), the size of the support set of the cluster (+ eTS), the contribution of the rule to the output error (SAFIS [163], GAP-RBF, D-FNN, GD-FNN), the combination of the age and the total number of activations (EFuNN, IBeM, FBeM, eGNN), or the minimal allowed distance between the cluster centers (ENFM). In [50], a cluster is removed from the model if no sample in a certain time interval rests within its bounds. The time interval is defined by the user. A drawback of this approach concerns long steady-state regimes. In this case, important clusters can be removed.

In D-FNN [196], GD-FNN [197], GAP-RBF [86], SAFIS [163] and SOFNN [121], cluster removing depends on the model output error. In D-FNN, an *error reduction ratio* is introduced to define the contribution of a certain local model to the overall output error. If the local model does not contribute significantly to the error reduction, the cluster is removed. A similar approach is addressed in GD-FNN [197]. In addition to an *error reduction ratio*, a *sensitivity index* is introduced. The clusters are removed according to these two values. In SAFIS [163], an estimation of the change in the output error is given when the cluster is removed from the model structure. If this value is higher than a threshold, the cluster is removed. SOFNN [121] introduces a procedure to remove clusters according to the concept of the *optimal brain surgeon approach* [83,122]. This approach is based on the sensitivity of the model output error according to the change of local model parameters. If the sensitivity is greater than a user defined threshold, the cluster is removed. Similar mechanisms were introduced in [196] and [197].

In [68] (Neural Gas), clusters are removed if they were generated $k - a_{\max}$ iterations before, where k stands for the current iteration, and $a_{\max}$ is a user-defined threshold.

In [29] (xTS), clusters are removed based on their support set and age. The support set is defined as the number of samples that belongs to a cluster. A sample always belongs to the closest cluster. The age of a cluster is defined as the ratio between the accumulated time of samples and the current time. Clusters are removed according to the ratio between the support set and the overall number of samples and the age of clusters. The same condition is also used in + eTS [19], where the condition of *utility* is also used. The utility is the ratio between the number of cluster activations and the time the cluster was added to the model. The cluster is removed when these values differ from the average value, where the confidence band is defined by the standard deviation.

DFKNN [80] removes clusters if their support sets are smaller than a minimum value. The minimal support set is a user-defined parameter. A second condition is based on a time interval in which it is required that at least one new sample is within the cluster, otherwise the cluster is removed.

In EFuNN [94,95], a cluster is removed due to the age and the sum of cluster activations. The age is defined as the number of samples from the creation of the cluster to the current iteration. If the age of the cluster is higher than a predefined threshold and its number of activations is less than the age of the cluster multiplied by a user defined constant in [0, 1], the cluster is removed. The eGNN approach in [114,116] is closely related.

In PANFIS [152], clusters are removed when they are inconsequential in terms of contributing to past outputs and possible future estimations. This can be reduced to a compact closed analytical form through u-fold numerical integration for any arbitrary probability density functions $p(x)$ of the input data manifold.

In eFuMo [59], the removing mechanism is a modification of that used in + eTS. It is based on the rate between the support set N_{p_i}, and the age of the cluster, a_i. The age of a cluster is defined as $a_i = k - k_i$, where k stands for the current time instant, and k_i is the number of samples from the time instant when the $i-th$ cluster was created. The minimal condition for the existence of the cluster is

$$\textbf{if } N_{trh} \geq N_{p_i}(a_{trh}), \textbf{ then } \text{remove cluster}, \tag{4}$$

where N_{trh} stands for the minimal number of samples in the cluster (the support set), a_{trh} is the threshold for the age of the cluster, and $N_{p_i}(a_{trh})$ is the value of the support set when it reaches the age threshold a_{trh}. All thresholds are defined by the user, but they have some commonly used default values. The condition to remove a cluster is given in the form of

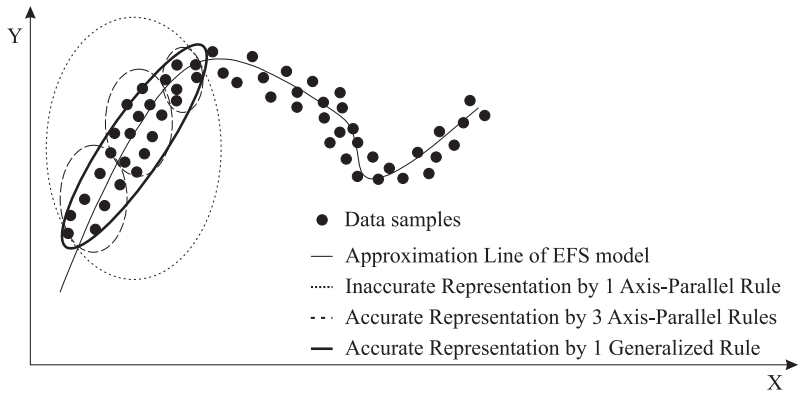


Fig. 7. Different representations of a one-dimensional approximation problem by conventional (axis-parallel, plotted with dotted/dashed ellipsoids) and a generalized (arbitrarily rotated) rule (plotted with a solid ellipsoid). Notice the more compact (and even more accurate) representation of the left-most upwards trend by the generalized rule.

the ratio between the support set and the age of the cluster as

$$\frac{N_{p_i}}{a_i} > \epsilon_{Na} \frac{1}{c} \sum_{i=1}^{c} \frac{N_{p_i}}{a_i}, \tag{5}$$

where ϵ_{Na} stands for a user-defined constant, which is less than one, and c is the number of clusters. All clusters that fulfill this condition remain in the model structure, whereas the others are removed.

In [109], the concept of *half-life* of a cluster or granule is introduced. Let

$$\Theta^i = 2^{(-\psi(h-h_a^i))} \tag{6}$$

be the activity factor associated to a cluster. The constant ψ is a decay rate, h the current time step, and h_a^i the last time step that the cluster was activated. Factor Θ^i decreases exponentially when h increases. The half-life of a cluster is the time spent to reduce the factor Θ^i by its half, that is, $1/\psi$. $1/\psi$ is a value useful to remove inactive clusters. Large values of ψ express lower tolerance to inactivity and higher privilege of more compact structures. Small values of ψ add robustness and prevent catastrophic forgetting. ψ is set in]0, 1[to keep model evolution active.

Although adding, removing, merging and splitting of clusters already assure some kind of homogeneity of clusters and model components and mostly more compact representations of data clouds, there are several key issues left for assuring i.) more elegant model architectures for higher predictive performance, ii.) more flexibility, iii.) higher robustness and iv.) handling of uncertainty v.) higher stability and better convergence, and vi.) higher computational speed and practical useability for model updates in order to greatly increase applicability of evolving (neuro-)fuzzy systems in real-world application scenarios. These will be handled in the next section, where each subsection addresses a particular issue as mentioned above in the same order (i.) to vi.)).

4. Advanced aspects and methodologies

4.1. Advanced architectures for increased performance and representation

Almost all of the aforementioned E(N)FS approaches employ the classical fuzzy model architecture regarding the antecedent space, which employs AND-connections of fuzzy sets in the single rules with the usage of a t-norm [102]. A rule is defined as follows

$$\text{Rule}_i\text{: IF } x_1 \text{ is } \mu_{i1} \text{ AND } \ldots \text{ AND } x_p \text{ is } \mu_{ip} \text{ THEN } y_i = f_i(x). \tag{7}$$

with p the dimensionality of the feature space, $\mu_{i1}, \ldots, \mu_{ip}$ linguistic terms (such as, e.g., high, intense, weak), formally represented by fuzzy sets [200], and $f_i(x)$ the consequent part, which can be a real value, a function or a class label. Through the AND-connections, rule activation levels can be obtained, which are typically normalized in the inference process and aggregated over all rules (through a t-conorm) to achieve a final model output — which is either a fuzzy set or a crisp value depending on whether a Mamdani-type or a Takagi-Sugeno type fuzzy system is applied.

The AND-connections in (7), when established through t-norms in order to achieve a rule activation level, induce axis-parallel rules. This prevents the possibility to model local correlations between input dimensions accurately and compactly. Either more rules are needed for an accurate representation of local data clouds or inaccurate representations of these are obtained. Fig. 7 gives a two dimensional example of this issue, with input variable X and output variable Y. The authors in [118] proposed the use of generalized versions of fuzzy rules in an evolving context. These are defined as

$$\text{Rule}_i\text{: IF } x \text{ IS (about) } \mu_i \text{ THEN } y_i = f_i(x). \tag{8}$$

μ_i denotes a high-dimensional kernel function, which, in accordance to the basis function networks spirit, is given by the multivariate Gaussian distribution

$$\mu_i(x) = \exp\left(-\frac{1}{2}(x - c_i)^T \Sigma_i^{-1}(x - c_i)\right), \tag{9}$$

with c_i the center and Σ_i^{-1} the inverse covariance matrix of the i-th rule, allowing an arbitrary rotation and spread of the rule. This generalized form of fuzzy rules has been also successfully used in GS-EFS [139] and PANFIS [152], where specific projection concepts have been developed in order to gain an equivalent axis-parallel rule base with conventional fuzzy sets, to maintain linguistic interpretability [200]. In [137], generalized rules have been successfully integrated in the all-pairs technique (EFC-AP, see Section 2.2) for a better representation of rules in multi-class classification problems. In [5], generalized rules have been used in evolving TS neuro-fuzzy classifiers employing classical single model architecture.

An extension of classical EFS architecture has been proposed in [104], which defines the consequent of rules as a weighted combination of mercer kernels. Therefore, LS-SVM can be applied in order to estimate the weights as support vectors in each local region, which may provide a better accuracy especially when there is an intrinsic local nonlinearity in the data.

In order to address uncertainty contained in data streams (or even in expert knowledge) on a second level appearance, e.g., fuzzy data samples which are influenced by noise, [92] proposed an *evolving type-2 fuzzy system* approach, termed as SEIT2-FNN. Type-2 fuzzy systems were invented by Lotfi Zadeh in 1975 [201] for the purpose of modeling the uncertainty in the membership functions of usual (type-1) fuzzy sets. Through the so-called footprint of uncertainty (FOU) [124], they are able to model such occurrences of second level uncertain fuzzy data. SEIT2-FNN uses classical interval-valued fuzzy sets, where the firing strength of type-2 fuzzy rules serves as motivation for rule and fuzzy set evolution. Thereby, this approach assures ϵ-completeness with ϵ being the threshold used for the minimal firing strength. It also embeds a fuzzy set reduction method for strongly overlapping sets. It applies a rule-ordered Kalman filter for consequent learning and an incremental gradient descent algorithm for antecedent learning.

Latter, other techniques for evolving type-2 fuzzy systems were suggested in [190] (eT2FIS), [178] (McIT2FIS), [157] (eT2RFNN) and in [156] (ST2Class) (for classification), which significantly expands the original approach in [92] with several concepts such as active learning for sample selection policies, curse of dimensionality reduction by feature weighting, and handling of cyclic drifts.

A new variant of neuro-fuzzy architecture has been proposed in [174], which has been termed evolving neofuzzy neuronal network (ENFN). ENFN splits the multi-dimensional input space to single uni-variate rules, which therefore reduces error-proneness of the model due to curse of dimensionality effects on structural basis in a natural way (see Section 4.3). Even more important, the inference process and the learning is completely independent from the number of inputs; the former just applies the sum of functional activations of each single rule (thus, over all inputs) to a combined output. The functional activation of a single rule is given by a weighted average of activations of two fuzzy sets more adjacent to the current query sample, where the weights are incrementally learned from the data. New membership functions are created whenever the local error exceeds the mean over the global error plus its standard deviation. ENFN also removes unnecessary membership functions due to inactiveness [131].

Multi-model classifiers, as discussed in Section 2.2, can also be seen as advanced architectures, achieving less class imbalance due to class-decomposition and using advanced techniques (from preference relation theory) for combining the outputs of evolving models as weak classifiers.

Furthermore, recently evolving deep (fuzzy) rule-based classifiers have been proposed in [27]. They are based on the autonomous multi-model systems architecture (ALMMo) [28] and avoid the limitations of current deep learning neural networks structures, which: i) are usually completely un-interpretable (apart from some hierarchical feature representations with different zooms in the case of context-based image data); and ii) require very high computational efforts in batch off-line training cycles.

4.2. Drift handling for increased flexibility

In predictive analytics and machine learning, *concept drift* means that the statistical properties of either the input or target variables change over time in unforeseen ways. In particular, drifts either denote changes in the underlying data distribution (input space drift), in the underlying relationship between model inputs and targets (joint drift) or in the prior probabilities of the target class resp. in the distribution of the target vector (target concept drift) — see [100] for a recent comprehensive survey discussing several variants of drifts. Drifts can happen because the system behavior, environmental conditions or process states dynamically change during the online learning process, which makes the (input/output) relations and dependencies contained in and modeled from the old data samples — more and more obsolete as time passes.

As already pointed out at the beginning of Section 3, evolving modeling techniques are an adequate methodology to handle drifts. When the drift is intense enough (abrupt drifts, shifts), new model components (rules) are typically evolved automatically by the evolving concepts discussed above. Such automatic handling within the learning procedure is also referred as *passive drift handling* [100], which abandons the necessity of detecting drifts explicitly. On the other hand, drifts may also be of lower intensity or of gradual nature [71], which typically deteriorates the local rules and hence overall performance [81].

The pioneering study to handle such drift cases is [131]. The basic idea concerns increasing the flexibility of the parameter updates through forgetting concepts. Forgetting is achieved through exponentially outweighing older samples over time with the use of a factor, whose value can be adapted according to the intensity of a drift, measured with the use of the *concept of rule ages* [19]. Forgetting of both, antecedent and consequent parameters in EFS was performed in [131] for achieving increased flexibility (of eTS+ and FLEXFIS) and thus significantly increased performance on several real-world (drifting) data sets. Many other EF(N)S methods also include the idea of forgetting older samples, but, typically, solely in the consequent parameters when being updated through recursive (fuzzily) weighted least squares (RFWLS) [10] (an exception is the eFuMo approach [59,202], which also performs forgetting in the antecedent space). The RFWLS technique proposed in [10] is fundamental in many E(N)FS methods that rely on the update of linear consequent parameters (see [140]).

Handling of *local drifts*, which are drifts that may appear with different intensities in different parts of the feature space (thus affecting different rules with varying intensity) is considered in [173]. The idea of this approach is that different forgetting factors are used for different rules instead of a global factor. This steers the local level of flexibility of the model. Local forgetting factors are adapted according to the local drift intensity (elicited by a modified variant of the Page-Hinkley test [146]) and the contribution of the rules in previous model errors.

Another form of drift is the *cyclic drift*, where changes in the (input/target) data distribution may happen at a certain point of time, but, latter, older distributions are revisited. ENFS approaches to deal with such drift cases were addressed in [156,157] using type-2 recurrent (neuro-)fuzzy systems, termed as eT2RFNN. The idea is to prevent re-learning of older local distributions from scratch and thus to increase the early significance of the rules.

Whenever a drift cannot be explicitly detected nor it implicitly triggers the evolution of a new rule/neuron, a posteriori *drift compensation* is a promising option in order to (back-)improve the accuracy of the rules. This can be achieved through incremental rule splitting [143]. 'Blown-up' rules with high local errors and high volumes are split into two smaller ones along the main principal component axis (with the highest eigenvalue).

4.3. Curse of dimensionality and over-fitting avoidance

High dimensionality of the data stream mining and modeling problem becomes apparent whenever a larger variety of features and/or system variables are recorded, e.g., in multi-sensor networks, which characterize the dependencies and interrelations contained in the system/problem to be modeled. Depending on the ratio between the number of samples (seen so far) and the number of input dimensions, the curse of dimensionality may become apparent, which usually causes significant over-fitting effects [171] and thus affects the whole performance of the model. This is especially the case for models including localization components (granules) as is the case of E(N)FS (in terms of rules/neuron) [147,148], because in high-dimensional spaces, one cannot talk about locality any longer (on which these types of models rely) as all samples are moving to the edges of the joint feature space — see the analysis in Chapter 1 in [84].

Therefore, the reduction of the dimensionality is highly desired. In a data-stream modeling context, the goal is ambitious and much more sophisticated than in batch learning, because, as in case of changing/drifting data distributions, the importance of features for explaining the target may also change over time. This may be reflected in the ranks or weights of the features. A first study on online dimension reduction in a data stream context is proposed in [19] (eTS+), where the contribution of the features in the consequents of the rules is measured in terms of their gradients in the hyper-planes: those features whose contribution over all rules is negligible can be discarded. Thus, this approach performs a crisp feature selection, but does not respect the possibility that some features may become important again at a later stage, thus should be also reactivated in the model. The same consideration holds to the approach in [153] (GENEFIS), which extends the approach in [19] by also integrating the contribution of the features in the antecedent space (regarding their significance in the rules premise parts). In [4], online crisp feature selection was extended to a local variant, where for each rule a separate feature (importance) list was incrementally updated. This was useful to achieve more flexibility due to *local feature selection* characteristics. Features become differently important in different parts of the feature space. A new design of the fuzzy inference model for predicting new samples was required and a solution presented.

To overcome a crisp selection and to offer feature reactivation, the approach in [133] proposes the incremental learning of *feature weights* $\in [0, 1]$, where a weight close to 1 denotes that the feature is important and a weight close to 0 that it is unimportant. By updating the features weights with single samples (achieved through an incremental version of Dy and Brodley's separability criterion [60]), slight changes in the weights are achieved over time. They prevent an abrupt inclusion or exclusion of features. Therefore, a feature is able to become reactivated automatically through weight updating, because the model is always learnt on the same whole feature space (thus no input structure changes in the model are needed, which requires time-intensive re-training phases). Curse of dimensionality reduction is then achieved i) by integrating the weights in the incremental learning procedure to down-weigh the contributions of unimportant features in rule evolution criteria and parameter updates, ii) by integrating the weights in the inference process when producing predictions on new samples to down-weigh the contributions of unimportant features to the final model output, and iii) when showing the learnt model to the experts/operators (by simply discarding features with low weights in the antecedents and consequent parts of the rules). The approach in [133] handles classification problems and designs the incremental feature weighting method for evolving fuzzy classifiers using single-model and all-pairs architectures (see Section 2.2). In [139], the feature weighting concepts have been adopted to the regression case where a re-scaled Mahalanobis distance measure is developed to integrate weights in consistent and monotonic fashion (i.e., that lower weights in fact always induce lower distances).

Another possibility for a smooth input structure change is proposed in [144] for regression problems with the use of partial least squares (PLS). PLS performs a transformation of the input space into latent variables (LVs) by maximizing the covariance structure between inputs and the target [77]. The coordinate axes are turned into a position (achieving latent variables as weighted linear combination of the original ones) that allows to better explain the complexity of the modeling problem. Typically, a lower number of LVs is needed to achieve an accurate regression. Scores on the lower number of LVs (projected samples) are used as input in the evolving models. LVs are updated incrementally with new incoming samples based on the concept of Krylov sequences in combination with Gram-Schmidt orthogonalization. Previous works in [45,62,98] also perform an incremental update of the LV space for evolving models, but using unsupervised principal component analysis (PCA) [90].

4.4. Uncertainty and reliability

Uncertainty arises during modeling whenever i) either data is affected significantly by noise or is not dense enough (statistically insignificant), especially at the start of the learning process; and ii) the input by humans (in the form of fuzzy rules) is vague due to limited expertise level or forms of cognitive impairments, e.g., distraction, fatigue, boredom, tiredness. Uncertainty is also an important aspect especially during the inference process when predicting and/or classifying new samples in order to indicate how reliable the model and its predictions are. For instance, in a classification system, the certainty of the predictions may support/influence the users/operators in a final decision.

The pioneering approach for achieving *uncertainty in evolving fuzzy modeling* for regression problems was proposed in [180]. The approach is deduced from statistical noise and quantile estimation theory. The idea is to find a lower and an upper fuzzy function for representing a confidence interval, i.e.,

$$\underline{f}(x_k^*) \leq f(x_k^*) \leq \overline{f}(x_k^*) \qquad \forall k \in \{1,\ldots,N\}, \tag{10}$$

with N the number of data samples seen so far. The main requirement is to define the band to be as narrow as possible and to contain a certain percentage of the data. This is based on the calculation of the expected covariance of the residual between the model output and new data in local regions as modeled by a linear hyper-plane. The following formulas for the local error (j-th rule) were obtained in [180] after deductions and reformulations,

$$\overline{\underline{f}}_j(x_k^*) = \Psi_j(x_k^*) l_j(x_k^*) \pm t_{\alpha,\Sigma(N)-deg}\hat{\sigma}\sqrt{(x_k^*\Psi_j(x_k^*))^T P_j(\Psi_j(x_k^*)x_k^*)}, \tag{11}$$

where $t_{\alpha,\Sigma(N)-deg}$ stands for the percentile of the t-distribution for $100(1-2\alpha)$ percentage confidence interval (default $\alpha = 0.025$) with $\Sigma(N) - deg$ degrees of freedom, and P_j denotes the inverse Hessian matrix. deg denotes the degrees of freedom in a local model. The symbol $\hat{\sigma}$ is the variance of model errors and the first term denotes the prediction of the jth local rule. The sum over all f_j's before the $\pm$ symbol refers to the conventional TS fuzzy model output, with $\Psi_j(.)$ the normalized membership degree and l_j the consequent function of the j-th rule. The term after $\pm$ provides the output uncertainty for the current query sample x_k^*.

Another approach to address uncertainty in model outputs is proposed in [112,113], where fuzzy rule consequents are represented by two terms, a linguistic – containing a fuzzy set (typically of trapezoidal nature) – and a functional – as in the case of TS fuzzy systems. The linguistic term offers a direct fuzzy output, which, according to the widths of the learned fuzzy sets, may reflect more or less uncertainty in the active rules (i.e., those rules which have non-zero or at least ϵ membership degree). A granular prediction is given by the convex hull of those sets which belong to active rules. The width of the convex hull can be interpreted as confidence intervals and given as final model output uncertainty. Evolving granular methods were successfully applied to financial time-series forecasting [110], Parkinson's telemonitoring [114], control of chaotic systems [115], rainfall prediction [113] and autonomous robot navigation [111]. In particular, evolving control design and closed-loop Lyapunov stability using an evolving granular model and evolving fuzzy controller in the loop are achieved in [115].

Uncertainty in classification problems using evolving fuzzy classifiers is addressed in [134], see subsequent section. The confidence in predicted class labels is given by a combination of: i) the closeness of the sample to the decision boundary (the closer, the more ambiguous the final classification statement); ii) class overlap degrees (the more overlap, the more ambiguous the final class), and iii) the novelty content calculated through the concept of ignorance (the higher the novelty is, the higher the unreliability is in the final class). A confidence vector is delivered in addition to the class label, representing the confidences in all classes. These concepts are also applied for all-pairs classification: i) by integrating confidence levels of pair-wise classifiers in a preference relation matrix, see Section 2.2); and ii) calculating the difference in certainty between the most and second most supported class. It is interesting to notice that novelty content is also implicitly handled in the error bars in [180] (see Eq. (11)) as for samples lying in extrapolation regions, the statistically motivated error bars are wider.

Apart from model uncertainty, *parameter uncertainty* can be an important aspect when deciding whether the model is stable and robust. Especially in the cases of insufficient or poorly distributed data, parameter uncertainty typically increases. Parameter uncertainty in EFS has been represented in [142,181] in terms of the use of the Fisher information matrix [65], with the help of some key measures extracted from it. In [181], parameter uncertainty is used for guiding the design of experiments process in an online incremental manner. In [142], it is used for guiding online active sample selection.

4.5. Stability and convergence

Stability of the learning algorithms and the resulting models is an important issue (as one part of the stability-plasticity dilemma) in order to be able to guarantee accurate and robust model predictions during online execution based on new incoming data stream samples. For instance, there should not be any predictions of target values which are widely out of the target's range and thus denote an unrealistic state, which may harm the system when it is internally further processed.

An issue which is closely related to the stability of models and their predictions is the convergence of the model parameters to a kind of optimality in the objective function sense, e.g., often in the least squares (LS) sense (especially, when optimizing consequent parameters, least squares is the most prominent choice, see above). When using the recursive (fuzzily weighted) least squares approach, as being done by many of the current techniques, convergence to optimality is guaranteed as long as there is no structural change [127], i.e. no neurons or rules are evolved and/or moved. FLEXFIS takes care of this issue and can achieve sub-optimality in the LS sense subject to a constant due to a decreasing learning gain in the cluster partitions.

Another and an even more direct way to guarantee stability is to show convergence of the model error itself, mostly to provide an upper bound on the (development of the) model error over time. This could be accomplished i.) in the PANFIS approach by the usage of an extended recursive LS method, which integrates a binary multiplication factor in the inverse Hessian matrix update which is set to 1 when the approximation error is bigger than the system error, and ii.) in SOFMLS by applying a modified least squares algorithm to train both, parameters and structures with the support of linearization and Lyapunov functions. It is remarkable that in this approach the evolution or pruning of rules does not harm the convergence of the parameters, as these only change the dimension of the inverse Hessian and the weight matrices. Based on SOFMLS approach, the bound on the identification error could be made even smaller in [166] with the usage of an own designed dead-zone recursive least square algorithm. SEIT2FNN employs a heuristic-based approach by resetting the inverse Hessian matrix to q^*I with I the identity matrix after several update iterations of the consequent parameters. This resetting operation keeps the inverse Hessian matrix bounded and helps to avoid divergence of parameters.

In [115], an evolving controller approach was designed based on neuro-fuzzy model architecture, and by proving a stability theorem using bounded inputs from linear matrix inequalities and parallel distributed computing concepts. This guarantees robust parameter solutions becoming convergent over time and omitting severe disturbances in control actions.

4.6. Online active learning and design of experiments

Most of the aforementioned ENFS methods require supervised samples in order to guide the incremental and evolving learning mechanisms into the right direction, and to maintain their predictive performance. This is especially true for the recursive update of consequent parameters and input/output product-space clustering. Alternatively, predictions may be used by the update mechanisms to reinforce the model. However, erroneous and imprecise predictions may spread, sum up over time and deteriorate model performance [170].

The problem in today's industrial systems with increasing complexity is that target values may be costly or even impossible to obtain and measure. For instance, in decision support and classification systems, ground truth labels of samples (from a training set, historic data base) have to be gathered by experts or operators to establish reliable and accurate classifiers, which typically require time-intensive annotation and labeling cycles [30,141]. Within a data stream mining process, this problem becomes even more apparent as experts have to provide a ground truth feedback quickly to meet real-time demands.

Therefore, it is important to decrease the number of samples for model update using sample selection techniques: annotation feedbacks or measurements for only those samples are required, which are expected to maintain or increase accuracy. This task can be addressed by *active learning* [172], a technique where the learner itself has control over which samples are used to update the models [48]. However, conventional active learning approaches operate fully in batch mode by iterating multiple times over a data base.

To select the most appropriate samples from data streams, *single-pass active learning (SP-AL)* for evolving fuzzy classifiers has been proposed in [134]. It relies on the concepts of *conflict* and *ignorance* [87]. The former addresses the degree of uncertainty in the classifier decision in terms of the class overlaping degree considering the local region where a sample falls within and in terms of the closeness of the sample to the decision boundary. The latter addresses the degree of novelty in the sample. A variant of SP-AL is given in [178] [177], where a *meta-cognitive* evolving scheme that relies on the concepts of *what-to-learn, when-to-learn* and *how-to-learn* is proposed. The what-to-learn aspect is handled by a sample deletion strategy, i.e., a sample is not used for model updates when the knowledge in the sample is similar to that of the model. Meta-cognitive learning has been further extended in [158] to regression problems (with the use of a fuzzy neural network architecture) and in [155] with the integration of a budget-based selection strategy. Such a budget-based learning was demonstrated to be of great practical usability as it explicitly constrains sample selection to a maximal allowed percentage.

In case of regression problems, permanent measurements of the targets can also be costly, e.g. in chemical or manufacturing systems that require manual checking of product quality. Therefore, online active learning for regression has been proposed in [142] for evolving generalized fuzzy systems (see Section 4.1) using GS-EFS, which relies on: i) the novelty of

Table 1
List of abbreviations and meanings.

Abbreviation	Meaning
GNG	Growing Neural Gas
ESOM	Evolve Self-organizing Maps
DENFIS	Dynamic Evolving Neural-Fuzzy Inference System
FLEXFIS	Flexible Fuzzy Inference Systems
GS-EFS	Generalized Smart Evolving Fuzzy Systems
EFuNN	Evolving Fuzzy Neural Network
D-FNN	Dynamic Fuzzy Neural Network
GD-FNN	Genetic Dynamic Fuzzy Neural Network
SAFIS	Sequential Adaptive Fuzzy Inference System
SCFNN	Self-Constructing Fuzzy Neural Network
RAN	Resource Allocating Network
GCS	Growing Cell Structure
SONFIN	Self Constructing Neural Fuzzy Inference Network
eTS	Evolving Takagi-Sugeno
DFKNN	Dynamic Fuzzy K-Nearest Neighbors
NeuroFAST	Neuro Function Activity Structure and Technology
GAP-RBF	Growing and Pruning Radial Basis Function
NFCN	Neural Fuzzy Control Network
ENFM	Evolving Neuro-Fuzzy Model
SOFNN	Self Organizing Fuzzy Neural Network
SOFMLS	Online Self-Organizing Fuzzy Modified Least-Squares Network
IBeM	Interval-Based Evolving Modeling
FBeM	Fuzzy set Based Evolving Modeling
eGNN	Evolving Granular Neural Networks
eFuMO	Evolving Fuzzy Model
RDE	Recursive Density Estimation
GANFIS	Generalized Adaptive Neuro-Fuzzy Inference Systems
NFCN	Neural Fuzzy Control Network
PANFIS	Parsimonious Network based on Fuzzy Inference System
RIVMcSFNN	Recurrent Interval-Valued Metacognitive Scaffolding Fuzzy Neural Network
eT2RFNN	Evolving Type-2 Recurrent Fuzzy Neural Network
SEIT2-FNN	Self-evolving Interval Type-2 Fuzzy Neural Network
ENFN	Evolving Neo-Fuzzy Neural Network
RBF	Radial Basis Function models
ANFIS	Adaptive Network-based Fuzzy Inference System
ESOM	Evolving Self-Organizing Map
ID3	Iterative Dichotomizer 3
ID4	Iterative Dichotomizer 4
ID5R	Incremental Decision Tree
LaSVM	Online Support Vector Machine
AnYa	Angelov and Yager system
TEDAClass	Typically and Eccentricity based Data Analytics Classifier
TEDA	Typically and Eccentricity based Data Analytics

a sample (ignorance); ii) the predicted output uncertainty measured in terms of local errors (see Section 4.4); and iii) the reduction of parameter uncertainty measured by the change in the E-optimality of the Fisher information matrix [65].

In summary, it is not only a matter of deciding if targets should be measured/labeled for available samples, but which samples in the input space should be gathered. Should the model expand its knowledge or increase significance of its parameters? Techniques from the field of design of experiments (DoE) [64,72] has been proposed. The pioneering online method for E(N)FS has been proposed in [181]. It relies on a combination of pseudo-Monte Carlo sampling algorithm (PM-CSA) [82] and max-min optimization criterion based on uniformly generated samples that satisfy a membership degree criterion for the worst local model.

Finally, in the next section, future directions will be provided which we see as essential for making evolving (neuro-)fuzzy systems applicable to a broader field of applications, especially for Big data and incomplete samples challenges, for making them even more robust and convergent as well as for reducing computational demands further.

5. Future directions

A variety of methods has been proposed over the last 15 years to guide the development and incremental adaptation of rule-based and neuro-fuzzy models from data streams. Interesting and persuasive practical solutions have been achieved. Nonetheless, propositions, lemmas, theorems and assurance that certain conditions will be fulfilled are still lacking in large parts (with a few exceptions — see Section 4.5) in the field of evolving clustering and evolving neuro-fuzzy and rule-based modeling from data streams. For instance, necessary and sufficient conditions to guarantee short term adaptation and long term survivability are still to be found. This is a major challenge because it will require the formalization of concept shift

Table 2
List of abbreviations and meanings (continuation).

Abbreviation	Meaning
EFC-AP	Evolving Fuzzy Classifier using All-Pairs Technique
ALMMo	Autonomous Multi-Model Systems Architecture
pClass	Parsimonious Classifier
pEnsemble	Parsimonious Ensemble
McIT2FIS	Meta-cognitive Interval Type-2 Neuro-fuzzy Inference System
MSAFIS	Modified Sequential Adaptive Fuzzy Inference System
eGM	Evolving Granulation Method
SOM	Self-Organizing Maps
ART	Adaptive Resonance Theory
ECM	Evolving Clustering Method
GK	Gustafson-Kessel clustering
eGKL	Evolving Gustafson-Kessel Like
eGKPCM	Evolving Gustafson-Kessel Possibilistic C-Means clustering
ELM	Evolving Local Mean
ARTMAP	Adaptive Resonance Theory
BIC	Bayesian Information Criterion
MSE	Mean Square Error
GRBF	Generalized Radial Basis Function
EBF	Ellipsoidal Basis Function
SVM	Support Vector Machine
KNN	K-Nearest Neighbor
eClass	Evolving Classifier
FRB	Fuzzy Rule Based
MIMO	Multi-Input Multi-Output
SGD	Stable Gradient Descent Algorithm

and concept drift, and to show how they affect searches in a hypothesis space from the point of view of simultaneous parameter estimation and structural adaptation. Systematic and formal approaches to deal with the stability-plasticity trade-off to ensure short-term adaptation and long term survivability are still lacking.

Missing data are common in real-world applications. They arise due to incomplete observations, transfer problems, malfunction of sensors, or incomplete information obtained from experts or on public surveys. The missing data issue is still an open topic in non-stationary data stream environments, in spite of having been extensively investigated in off-line settings. Particular sequences of missing data may cause instability of Lyapunov-stable closed-loop control systems and a loss of memory in evolving fuzzy and neuro-fuzzy modeling.

Further issues that remain unsatisfactorily addressed in the literature concerns characterization, design of experimental setups, and construction of workflows to guide development, performance evaluation, testing, validation, and comparison of algorithms in non-stationary environments. The evolution of rough-set models, Dempster-Shafer models and also aggregation functions are also important topics to expand the current scope of the area. T and S-norms, Uni and null-norms, and averaging functions are generally chosen *a priori* and kept fixed during model evolution. Approaches to switch aggregation operators based on properties of the data and to adapt associated parameters are still to be undertaken.

Large problems can often be divided into smaller ones, which can then be solved at the same time. Evolving systems in parallel high-performance computing will be explored in the following years. The rule-base modular and granular structure of fuzzy models is an interesting aspect to be exploited in high-frequency online applications. Moreover, a variety of particularities of different applications and evolution aspects in hardware with low resources (aiming for smart evolving models) are still to be addressed.

6. Conclusion

We presented a survey on evolving intelligent systems for regression and classification with emphasis on fuzzy and neuro-fuzzy methods. In-depth analyses of research contributions, especially over the last 15 years, which are fundamental to the current state-of-the-art of the field were discussed. The objective is guiding the readers to a clear understanding of the past and current challenges and relevant issues in the area. The survey discussed various evolution mechanisms such as adding, removing, merging and splitting clusters and local models in real-time. We highlighted open or partially addressed research directions, which we believe will help future investigations and developments in the area. (Tables 1, 2)

Acknowledgement

Igor Škrjanc, Jose Antonio Iglesias and Araceli Sanchis would like to thank to the Chair of Excellence of Universidad Carlos III de Madrid, and the Bank of Santander Program for their support. Igor Škrjanc is grateful to Slovenian Research Agency with the research program P2-0219, Modeling, simulation and control. Daniel Leite acknowledges the Minas Gerais Foundation for Research and Development (FAPEMIG), process APQ-03384-18. Igor Škrjanc and Edwin Lughofer acknowledges the support by the "LCM — K2 Center for Symbiotic Mechatronics" within the framework of the Austrian COMET-K2 program.

Fernando Gomide is grateful to the Brazilian National Council for Scientific and Technological Development (CNPq) for grant 305906/2014-3.

References

[1] C. Aggarwal, Data Streams: Models and Applications, Springer, New York, NY, USA, 2007.
[2] R. Agrawal, R. Bala, Incremental bayesian classification for multivariate normal distribution data, Pattern Recognit. Lett. 29 (13) (2008) 1873–1876.
[3] H. Ahn, Y. Chen, K. Moore, Iterative learning control: brief survey and categorization, IEEE Trans. Syst., Man, Cybern.Part C 37 (6) (2007) 1099–1120.
[4] S.A. Alizadeh, A. Kalhor, H. Jamalabadi, B. Araabi, M. Ahmadabadi, Online local input selection through evolving heterogeneous fuzzy inference system, IEEE Trans Fuzzy Syst. 24 (6) (2016) 1364–1377.
[5] A. Almaksour, E. Anquetil, Improving premise structure in evolving takagi-sugeno neuro-fuzzy classifiers, Evolv. Syst. 2 (2011) 25–33.
[6] G. Andonovski, G. Mušič, S. Blažič, I. Škrjanc, Online evolving cloud-based model identification for production control, IFAC-PapersOnLine 49 (5) (2016) 79–84.
[7] G. Andonovski, P. Angelov, S. Blažič, I. Škrjanc, A practical implementation of robust evolving cloud-based controller with normalized data space for heat-exchanger plant, Appl. Soft Comput. 48 (2016) 29–38.
[8] G. Andonovski, B.S.J. Costa, S. Blažič, I. Škrjanc, Robust evolving controller for simulated surge tank and for real two-tank plant, at - Automatisierungstechnik 66 (9) (2018) 725–734.
[9] G. Andonovski, G. Mušič, S. Blažič, I. Škrjanc, Evolving model identification for process monitoring and prediction of nonlinear systems, Eng. Appl. Artif. Intell. 68 (2018) 214–221.
[10] P. Angelov, D. Filev, An approach to online identification of takagi-sugeno fuzzy models, IEEE Trans. Syst. Man Cybern. - Part B 34 (1) (2004) 484–498.
[11] P. Angelov, An approach for fuzzy rule-base adaptation using online clustering, Integr. Method. Hybrid Syst. 35 (3) (2004) 275–289.
[12] P. Angelov, An approach for fuzzy rule-base adaptation using online clustering, Int. J. Approx Reason 35 (3) (2004) 275–289.
[13] P. Angelov, N. Kasabov, Evolving computational intelligence systems, in: Proc. 1st Int. WS on Genetic Fuzzy Systems, 76–82, Granada, Spain, 2005.
[14] P. Angelov, D. Filev, *Simpl_eTS*: a simplified method for learning evolving takagi-sugeno fuzzy models, Proc IEEE Int Conf on Fuzzy Systems, pp. 1068–1073, 2005.
[15] P. Angelov, X. Zhou, F. Klawonn, Evolving fuzzy rule-based classifiers, in: Computational Intelligence in Image and Signal Processing, 2007. CIISP 2007. IEEE Symposium on, IEEE, 2007, pp. 220–225.
[16] P. Angelov, X. Zhou, Evolving fuzzy-rule-based classifiers from data streams, IEEE Trans. Fuzzy Syst. 16 (6) (2008) 1462–1475.
[17] P. Angelov, Evolving takagi-sugeno fuzzy systems from streaming data (ets+), Evolv. Intell. syst. (2010) 21–50.
[18] P. Angelov, D. Filev, N. Kasabov, P. Angelov, D. Filev, N. Kasabov, Evolving Intelligent Systems: Methodology and Applications, Wiley-IEEE Press, New Jersey, 2010.
[19] P. Angelov, Evolving Intelligent Systems: Methodology and Applications, in: Evolving Takagi-Sugeno Fuzzy Systems From Streaming Data (eTS+), New Jersey: Wiley-IEEE Press, 2010, pp. 21–50.
[20] P. Angelov, D. Filev, N. Kasabov, Editorial, Evolv. Syst. 1 (1) (2010) 1–2.
[21] P. Angelov, R. Yager, Simplified Fuzzy Rule-based Systems Using Non-Parametric Antecedents and Relative Data Density, IEEE Workshop on Evolving and Adaptive Intelligent Systems (EAIS), pp. 62–69, 2011.
[22] P. Angelov, P. Sadeghi-Tehran, R. R., An approach to automatic real-time novelty detection, object identification, and tracking in video streams based on recursive density estimation and evolving takagi-sugeno fuzzy systems, Int. J. Intell. Syst. 26 (3) (2011) 189–205.
[23] P. Angelov, Autonomous Learning Systems: From Data Streams to Knowledge in Real-Time, Wiley, 2012.
[24] P. Angelov, Evolving rule-based models: a tool for design of flexible adaptive systems, Physica 92 (2013).
[25] P. Angelov, Anomaly detection based on eccentricity analysis, IEEE Symp. Evolv. Autonomous Learn. Syst. (EALS) (2014) 1–8.
[26] P. Angelov, I. Škrjanc, S. Blažič, A robust evolving cloud-based controller, in: J. Kacprzyk, W. Pedrycz (Eds.), Springer Handbook of Computational Intelligence, (Springer handbooks), Springer, Berlin; Heidelberg:, 2015, pp. 1435–1449.
[27] P. Angelov, X. Gu, A cascade of deep learning fuzzy rule-based image classifier and SVM, Proc of the IEEE Int Conf on Systems, Man, and Cybernetics (SMC2017), IEEE, 2017, pp. 746–751.
[28] P. Angelov, X. Gu, J. Principe, Autonomous learning multi-model systems from data streams, IEEE Trans. Fuzzy Syst. (2018), doi:10.1109/TFUZZ.2017.2769039.
[29] M. Asif, P. Angelov, H. Ahmed, An approach to real-time color-based object tracking, Proceedings of International Symposium on Evolving Fuzzy Systems 2006, 2006, pp. 86–91.
[30] A. Azcarraga, M.H. Hsieh, S.L. Pan, R. Setiono, Improved SOM labeling methodology for data mining applications, Soft Computing for Knowledge Discovery and Data Mining, NY: Springer, 2008. Pp. 45–75.
[31] M.F. Azeem, H. Hanmandlu, N. Ahmad, Structure identification of generalized adaptive neuro-fuzzy inference systems, IEEE Trans. Fuzzy Syst. 11 (5) (2003) 666–681.
[32] R.D. Baruah, P. Angelov, J. Andreu, Simpl_Eclass: simplified potential-free evolving fuzzy rule-based classifiers, IEEE Int Conf on Systems, Man, and Cybernetics (SMC), pp. 2249–2254, 2011.
[33] D. Baruah, P. Angelov, Evolving local means method for clustering of streaming data, IEEE Int Conf on Fuzzy Systems, pp. 1–8, 2012.
[34] C. Bishop, Pattern Recognition and Machine Learning, Springer, New York:, 2007.
[35] W. Black, P. Haghi, K. Arigur, Adaptive systems: history, Tech., Probl. Perspect., Syst. 2 (2014) 606–660.
[36] S. Blažič, I. Škrjanc, D. Matko, A robust fuzzy adaptive law for evolving control systems, Evolv. Syst. 5 (1) (2014) 3–10.
[37] S. Blažič, P. Angelov, I. Škrjanc, Comparison of approaches for identification of all-data cloud-based evolving systems, IFAC Conf on Embedded Systems, Computer Intelligence and Telematics CESCIT, pp. 129–134, Maribor, Slovenia, 2015.
[38] S. Blažič, I. Škrjanc, Problems of identification of cloud-based fuzzy evolving systems, Artif. Intell. Soft Comput. ICAISC 2016. Lect. Notes Comput. Sci. 9692 (2016) 173–182.
[39] A. Bordes, L. Bottou, The huller: a simple and efficient online Svm, in: European Conf on Machine Learning, Springer, 2005, pp. 505–512.
[40] G.A. Carpenter, S. Grossberg, J.H. Reynolds, Artmap: supervised real-time learning and classification of nonstationary data by a self-organizing neural network, Neural Netw. 4 (5) (1991) 565–588.
[41] G.A. Carpenter, S. Grossberg, N. Markuzon, J.H. Reynolds, D.B. Rosen, Fuzzy artmap: a neural network architecture for incremental supervised learning of analog multidimensional maps, IEEE Trans. Neural Netw. 3 (5) (1992) 698–713.
[42] G.A. Carpenter, S. Grossberg, Adaptive resonance theory, Springer, 2016.
[43] O. Castillo, P. Melin, Hybrid intelligent systems for time series prediction using neural networks, fuzzy logic, and fractal theory, IEEE Trans. Neural Netw. 13 (6) (2002) 1395–1408.
[44] G. Cauwenberghs, T. Poggio, Incremental and decremental support vector machine learning, in: Advances in neural information processing systems, 2001, pp. 409–415.
[45] C. Cernuda, E. Lughofer, P. Hintenaus, W. Märzinger, T. Reischer, M. Pawlicek, J. Kasberger, Hybrid adaptive calibration methods and ensemble strategy for prediction of cloud point in melamine resin production, Chemom. Intell. Lab. Syst. 126 (2013) 60–75.
[46] K.M.A. Chai, H.L. Chieu, H.T. Ng, Bayesian online classifiers for text classification and filtering, in: Proc ACM SIGIR Conf on Research and development in information retrieval, ACM, 2002, pp. 97–104.
[47] S.L. Chiu, Fuzzy model identification based on cluster estimation, J. Intell. Fuzzy Syst. 2 (1994) 267–278.
[48] D. Cohn, L. Atlas, R. Ladner, Improving generalization with active learning, Mach. Learn. 15 (2) (1994) 201–221.

[49] B. Costa, P. Angelov, L.A. Guedes, Fully unsupervised fault detection and identification based on recursive density estimation and self-evolving cloud-based classifier, Neurocomputing 150 (2015) 289–303.
[50] F. Crespo, R. Weber, A methodology for dynamic data mining based on fuzzy clustering, Fuzzy Sets Syst. 150 (2) (2005) 267–284.
[51] A.C. de Carvalho, A.A. Freitas, A Tutorial on multi-label classification techniques, in: Foundations of Computational Intelligence Volume 5, Springer, 2009, pp. 177–195.
[52] A. Declercq, J. Piater, Online learning of gaussian mixture models–a two-level approach, in: Proc 3rd Int Conf on Computer Vision Theory and Applications (VISAPP), Funchal, Portugal, 2008, pp. 605–611.
[53] D. Deng, N. Kasabov, ESOM: an algorithm to evolve self-organizing maps from online data streams, Proc. IEEE-INNS-ENNS 6 (2000) 3–8.
[54] D. Deng, N. Kasabov, Online pattern analysis by evolving self-organizing maps, Neurocomputing 51 (2003) 87–103.
[55] C. Domeniconi, D. Gunopulos, Incremental support vector machine construction, in: Data Mining, 2001. ICDM 2001, Proceedings IEEE International Conference on, IEEE, 2001, pp. 589–592.
[56] P.M. Domingos, G. Hulten, Catching up with the data: research issues in mining data streams, DMKD, 2001.
[57] D. Dovžan, I. Škrjanc, Recursive clustering based on a gustafson-kessel algorithm, Evolv. Syst. 2 (1) (2011) 15–24.
[58] D. Dovžan, I. Škrjanc, Recursive fuzzy c-means clustering for recursive fuzzy identification of time-varying processes, ISA Trans. 50 (2) (2011) 159–169.
[59] D. Dovžan, V. Logar, I. Škrjanc, Implementation of an evolving fuzzy model (eFuMo) in a monitoring system for a waste-water treatment process, IEEE Trans. Fuzzy Syst. 23 (5) (2015) 1761–1776.
[60] J. Dy, C. Brodley, Feature selection for unsupervised learning, J. Mach. Learn. Res. 5 (2004) 845–889.
[61] F. Ferrer-Troyano, J.S. Aguilar-Ruiz, J.C. Riquelme, Incremental rule learning based on example nearness from numerical data streams, in: Proc ACM Symp on Applied Computing, ACM, 2005, pp. 568–572.
[62] D. Filev, F. Tseng, Novelty detection based machine health prognostics, in: Proc. of the 2006 International Symposium on Evolving Fuzzy Systems, Lake District, UK, 2006, pp. 193–199.
[63] D. Filev, O. Georgieva, An extended version of the Gustafson–Kessel algorithm for evolving data stream clustering, Evolv. Intell. syst. (2010) 273–300.
[64] G. Franceschini, S. Macchietto, Model-based design of experiments for parameter precision: state of the art, Chem. Eng. Sci. 63 (19) (2008) 4846–4872.
[65] B. Frieden, R. Gatenby, Exploratory Data Analysis Using Fisher Information, Springer Verlag, New York:, 2007.
[66] H. Frigui, R. Krishnapuram, A robust algorithm for automatic extraction of an unknown number of clusters from noisy data, Pattern Recognit. Lett. 17 (12) (1996) 1223–1232.
[67] B. Fritzke, Growing cell structures: a self-organizing network for unsupervised and supervised learning, Neural Netw. 7 (9) (1994) 1441–1460.
[68] B. Fritzke, Advances in neural information processing systems 7, in: ch. A Growing Neural Gas Network Learns Topologies, Cambridge: MIT Press, 1995, pp. 625–632.
[69] B. Fritzke, A growing neural gas network learns topologies, Adv. Neural Inf. Proc. Syst. (7) (1995) 625–632.
[70] J. Gama, Knowledge discovery from data streams. chapman and Hall/CRC, Boca Raton, FL, USA, 2010.
[71] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, A. Bouchachia, A survey on concept drift adaptation, ACM Comput. Surv. 46 (4) (2014). P. article: 44,
[72] S. Garcia, A. Fernandez, J. Luengo, F. Herrera, Advanced nonparametric tests for multiple comparisons in the design of experiments in computational intelligence and data mining: experimental analysis of power, Inf. Sci. 180 (2010) 2044–2064.
[73] P. Geczy, Big data characteristics, Macrotheme Rev. 3 (6) (2014) 94–104.
[74] F. Gomide, Evolving Granular Neural Networks from Data Streams, Wiley Online Library, 2017, doi:10.1002/047134608X.W8358.
[75] G. Goodwin, K. Sin, Adaptive Filtering, Prediction, and Control, Prentice-Hall, Englewood Cliffs, N.J., USA,, 1984.
[76] D.E. Gustafson, W.C. Kessel, Fuzzy clustering with a fuzzy covariance matrix, in: IEEE Conf on Adaptive Processes, 1979, pp. 761–766.
[77] M. Haenlein, A. Kaplan, A beginner's guide to partial least squares (PLS) analysis, Understand. Stat. 3 (4) (2004) 283–297.
[78] P.M. Hall, Y. Hicks, A Method to Add Gaussian Mixture Models, University of Bath, 2004 Technical report csbu-2004-03.
[79] L. Hartert, M. Sayed-Mouchaweh, P. Billaudel, A Semi-Supervised Dynamic Version of Fuzzy k-Nearest Neighbours to Monitor Evolving Systems, Evolv. Syst. 1 (2010) 3–15.
[80] L. Hartert, M. Sayed-Mouchaweh, P. Billaudel, A semi-supervised dynamic version of fuzzy k-nearest neighbours to monitor evolving systems, Evolv. Syst. 1 (2010) 3–15.
[81] B. Hartmann, O. Banfer, O. Nelles, A. Sodja, L. Teslič, I. Škrjanc, Supervised hierarchical clustering in fuzzy model identification, IEEE Trans. Fuzzy Syst. 19 (6) (2011) 1163–1176.
[82] B. Hartmann, J. Moll, O. Nelles, C.P. Fritzen, Hierarchical local model trees for design of experiments in the framework of ultrasonic structural health monitoring, in: Proceedings of the IEEE International Conference on Control Applications, 2011, pp. 1163–1170.
[83] B. Hassibi, D.G. Stork, Second-order derivatives for network pruning: optimal brain surgeon, Adv. Neural Inf. Process. 4 (1993) 164–171.
[84] T. Hastie, R. Tibshirani, J. Friedman, The Elements of Statistical Learning: Data Mining, Inference and Prediction - Second Edition, Springer, New York Berlin Heidelberg, 2009.
[85] W. Ho, W. Tung, C. Quek, An Evolving Mamdani-Takagi-Sugeno Based Neural-fuzzy inference system with improved interpretability–accuracy, in: Proceedings of the WCCI 2010 IEEE World Congress of Computational Intelligence, Barcelona, pp. 682–689, 2010.
[86] G.B. Huang, P. Saratchandran, N. Sundararajan, A recursive growing and pruning RBF (GAP-RBF) algorithm for function approximations, in: Proc of The Fourth International Conf on Control and Automation (ICCA'03), IEEE, Montreal, 2003, pp. 491–495.
[87] E. Hüllermeier, K. Brinker, Learning valued preference structures for solving classification problems, Fuzzy Sets Syst. 159 (18) (2008) 2337–2352.
[88] J.A. Iglesias, A. Ledezma, A. Sanchis, Ensemble method based on individual evolving classifiers, Evolving and Adaptive Intelligent Systems (EAIS), 2013 IEEE Symposium on, pp. 56–61, 2013.
[89] J.A. Iglesias, A. Ledezma, A. Sanchis, An ensemble method based on evolving classifiers: estacking, Evolving and Autonomous Learning Systems (EALS), 2014 IEEE Symposium on, pp. 124–131, 2014.
[90] I. Jolliffe, Principal Component Analysis, Springer, New York, 2002.
[91] C.F. Juang, C.T. Lin, An online self-constructing neural fuzzy inference network and its applications, IEEE Trans. Fuzzy Syst. 6 (1) (1998) 12–32.
[92] C.F. Juang, Y. Tsao, A self-evolving interval type-2 fuzzy neural network with online structure and parameter learning, IEEE Trans. Fuzzy Syst. 16 (6) (2008) 1411–1424.
[93] D. Kangin, P. Angelov, J.A. Iglesias, A. Sanchis, Evolving classifier tedaclass for big data, Procedia Comput. Sci. 53 (2015) 9–18.
[94] N. Kasabov, T. Yamakawa, G. Matsumoto, Evolving Fuzzy Neural Networks-Algorithms, Applications and Biological Motivation, Methodologies for the Conceptation, Design and Application of Soft Computing, vol. 1., Japan: World Scientific, 271–274, 1998.
[95] N. Kasabov, Evolving fuzzy neural networks for supervised/unsupervised online knowledge-based learning, IEEE Trans. Syst. Man Cybern. - Part B 31 (6) (2001) 902–918.
[96] N. Kasabov, Q. Song, DENFIS: Dynamic evolving neural-fuzzy inference system and its application for time-series prediction, IEEE Trans. Fuzzy Syst. 10 (2) (2002) 144–154.
[97] N. Kasabov, Q. Song, Denfis: dynamic evolving neural-fuzzy inference system and its application for time-series prediction, IEEE Trans. Fuzzy Syst. 10 (2) (2002) 144–154.
[98] N. Kasabov, D. Zhang, P. Pang, Incremental learning in autonomous systems: evolving connectionist systems for online image and speech recognition, in: Proc of IEEE Workshop on Advanced Robotics and its Social Impacts, Hsinchu, Taiwan, 2005, pp. 120–125.
[99] N. Kasabov, Evolving connectionist systems, Springer, London, 2007.
[100] I. Khamassi, M. Sayed-Mouchaweh, M. Hammami, K. Ghedira, Discussion and review on evolving data streams and concept drift adapting, Evolv. Syst. 9 (1) (2017) 1–23.

[101] G. Klančar, I. Škrjanc, Evolving principal component clustering with a low run-time complexity for LRF data mapping, Appl. Soft Comput. J. 35 (2015) 349–358.
[102] E. Klement, R. Mesiar, E. Pap, Triangular norms, Dordrecht Norwell New York London:, Kluwer Academic Publishers, 2000.
[103] T. Kohonene, The self-organizing map, Proc. IEEE 78 (9) (1990) 1464–1480.
[104] M. Komijani, C. Lucas, B. Araabi, A. Kalhor, Introducing evolving takagi-sugeno method based on local least squares support vector machine models, Evolv. Syst. 3 (2) (2012) 81–93.
[105] R. Krishnapuram, J.M. Keller, A possibilistic approach to clustering, IEEE Trans. Fuzzy Syst. 1 (2) (1993) 98–110.
[106] M. Kuipers, P. Ioannou, Multiple Model Adaptive Control with Mixing, in: IEEE Transactions on Automatic Control, volume 55, 2010, pp. 1822–1836.
[107] D. Leite, P. Costa, F. Gomide, Interval-based evolving modeling, IEEE WS Evolv. Self-Develop. Intel Syst. (2009) 1–8.
[108] D. Leite, P. Costa, F. Gomide, Evolving granular neural network for semi-supervised data stream classification, Int. Joint Conf. Neural Netw.(IJCNN) (2010) 1–8.
[109] D. Leite, P. Costa, F. Gomide, Granular approach for evolving system modeling, in: Lecture Notes in Artificial Intelligence: Int. Conf. on Information Processing and Management of Uncertainty in Knowledge-Based Systems, Springer, Berlin, Heidelberg, 2010, pp. 340–349.
[110] D. Leite, F. Gomide, R. Ballini, P. Costa, Fuzzy granular evolving modeling for time series prediction, in: Proceedings of the IEEE International Conference on Fuzzy Systems, 2011, pp. 2794–2801.
[111] D. Leite, F. Gomide, Evolving linguistic fuzzy models from data streams, Combining Experimentation and Theory, Springer pp. 209–223, Berlin, Heidelberg, 2012.
[112] D. Leite, R. Ballini, P. Costa, F. Gomide, Evolving fuzzy granular modeling from nonstationary fuzzy data streams, in: Evolving Systems, volume 3, 2012, pp. 65–79.
[113] D. Leite, P. Costa, F. Gomide, Interval approach for evolving granular system modeling, in: Learning in Non-Stationary Environments, Springer, New York, NY, 2012, pp. 271–300.
[114] D. Leite, P. Costa, F. Gomide, Evolving granular neural networks from fuzzy data streams, Neural Netw. 38 (2013) 1–16.
[115] D. Leite, R. Palhares, V. Campos, F. Gomide, Evolving granular fuzzy model-based control of nonlinear dynamic systems, IEEE Trans. Fuzzy Syst. 23 (4) (2015) 923–938.
[116] D. Leite, M. Santana, A. Borges, F. Gomide, Fuzzy granular neural network for incremental modeling of nonlinear chaotic systems, IEEE Int. Conf. Fuzzy Syst. (FUZZ-IEEE) (2016) 64–71.
[117] V. Lemaire, C. Salperwyck, A. Bondu, A survey on supervised classification on data streams, in: European Business Intelligence Summer School, Springer, 2014, pp. 88–125.
[118] A. Lemos, W. Caminhas, F. Gomide, Multivariable gaussian evolving fuzzy modeling system, IEEE Trans. Fuzzy Syst. 19 (1) (2011) 91–104.
[119] A. Lemos, F. Gomide, W. Caminhas, Fuzzy evolving linear regression trees, Evolving Systems, volume 2, Springer, Heidelberg, 2011. 117–159
[120] A. Lemos, W. Caminhas, F. Gomide, S. Ramanna, L. Jain, R. Howlett, Evolving intelligent systems: methods, algorithms and applications, Emerging Paradigms in Machine Learning, 117–159, Springer, Berlin Heidelberg, Germany, 2013.
[121] G. Leng, G. Prasad, T.M. McGinnty, An online algorithm for creating self-organizing fuzzy neural networks, Neural Netw. 17 (10) (2004) 1477–1493.
[122] C.S. Leung, K.W. Wong, P.F. Sum, L.W. Chan, A pruning method for the recursive least squared algorithm, Neural Netw. 14 (2) (2001) 147–174.
[123] W. Li, H.H. Yue, S. Valle-Cervantes, S.J. Qin, Recursive PCA for adaptive process monitoring, J. Process Control 10 (5) (2000) 471–486.
[124] Q. Liang, J. Mendel, Interval type-2 fuzzy logic systems: theory and design, IEEE Trans. Fuzzy Syst. 8 (5) (2000) 535–550.
[125] C.T. Lin, A neural fuzzy control system with structure and parameter learning, Fuzzy Sets Syst. 70 (2–3) (1995) 183–212.
[126] F.J. Lin, C.H. Lin, P.H. Shen, Self-constructing fuzzy neural network speed controller for permanent-magnet synchronous motor drive, IEEE Trans. Fuzzy Syst. 9 (5) (2001) 751–759.
[127] L. Ljung, System identification: Theory for the user, Prentice Hall PTR, Prentic Hall Inc., Upper Saddle River, New Jersey, 1999.
[128] E. Lughofer, P. Angelov, X. Zhou, Evolving single-and multi-model fuzzy classifiers with flexfis-class, in: Fuzzy Systems Conference, 2007. FUZZ-IEEE 2007. IEEE International, IEEE, 2007, pp. 1–6.
[129] E. Lughofer, FLEXFIS: a robust incremental learning approach for evolving TS fuzzy models, IEEE Trans. Fuzzy Syst. 16 (6) (2008) 1393–1410.
[130] E. Lughofer, J.L. Bouchot, A. Shaker, Online elimination of local redundancies in evolving fuzzy systems, Evolv. Syst. (2) (2011) 165–187.
[131] E. Lughofer, P. Angelov, Handling drifts and shifts in online data streams with evolving fuzzy systems, Appl. Soft Comput. 11 (2) (2011) 2057–2068.
[132] E. Lughofer, Evolving fuzzy systems-methodologies, advanced concepts and applications, ser, in: J. Kacprzyk (Ed.), Studies in Fuzziness and Soft Computing Series, Berlin Heidelberg: Springer-Verlag, vol. 266, 2011.
[133] E. Lughofer, Online incremental feature weighting in evolving fuzzy classifiers, Fuzzy Sets Syst. 163 (1) (2011) 1–23.
[134] E. Lughofer, Single-pass active learning with conflict and ignorance, Evolv. Syst. 3 (4) (2012) 251–271.
[135] E. Lughofer, A dynamic split-and-merge approach for evolving cluster models, Evolv. Syst. 3 (3) (2012) 135–151.
[136] E. Lughofer, O. Buchtala, Reliable all-pairs evolving fuzzy classifiers, IEEE Trans. Fuzzy Syst. 21 (4) (2013) 625–641.
[137] E. Lughofer, E. Weigl, W. Heidl, C. Eitzinger, T. Radauer, Integrating new classes on the fly in evolving fuzzy classifier designs and its application in visual inspection, Appl. Soft Comput. 35 (2015) 558–582.
[138] E. Lughofer, M. Sayed-Mouchaweh, Autonomous data stream clustering implementing incremental split-and-merge techniques — towards a plug-and–play approach, Inf. Sci. 204 (2015) 54–79.
[139] E. Lughofer, C. Cernuda, S. Kindermann, M. Pratama, Generalized smart evolving fuzzy systems, Evolv. Syst. 6 (4) (2015) 269–292.
[140] E. Lughofer, Evolving fuzzy systems — fundamentals, reliability, interpretability and usability, in: P. Angelov (Ed.), Handbook of Computational Intelligence, World Scientific, New York, 2016, pp. 67–135.
[141] E. Lughofer, R. Richter, U. Neissl, W. Heidl, C. Eitzinger, T. Radauer, Explaining classifier decisions linguistically for stimulating and improving operators labeling behavior, Inf. Sci. 420 (2017) 16–36.
[142] E. Lughofer, M. Pratama, Online active learning in data stream regression using uncertainty sampling based on evolving generalized fuzzy models, IEEE Trans. Fuzzy Syst. 26 (1) (2018) 292–309.
[143] E. Lughofer, M. Pratama, I. Škrjanc, Incremental rule splitting in generalized evolving fuzzy systems for autonomous drift compensation, IEEE Trans. Fuzzy Syst. 26 (4) (2018) 1854–1865.
[144] E. Lughofer, A.C. Zavoianu, R. Pollak, M. Pratama, P. Meyer-Heye, H. Zörrer, C. Eitzinger, J. Haim, T. Radauer, Self-adaptive evolving forecast models with incremental PLS space updating for online prediction of micro-fluidic chip quality, Eng. Appl. Artif. Intell. 68 (2018) 131–151.
[145] M. Mermillod, A. Bugaiska, P. Bonin, The stability-plasticity dilemma: investigating the continuum from catastrophic forgetting to age-limited learning effects, Front Psychol. 4 (504) (2013).
[146] H. Mouss, D. Mouss, N. Mouss, L. Sefouhi, Test of page-hinkley, an approach for fault detection in an agro-alimentary production system, in: Proc of the Asian Control Conf, Vol 2, 2004, pp. 815–818.
[147] W. Pedrycz, F. Gomide, Fuzzy systems engineering: Toward human-Centric computing, John Wiley & Sons, Hoboken, New Jersey:, 2007.
[148] W. Pedrycz, A. Skowron, V. Kreinovich, Handbook of Granular Computing, John Wiley & Sons, Chichester, West Sussex, England:, 2008.
[149] J. Platt, A resource allocating network for function interpolation, Neural Computat. 3 (2) (1991) 213–225.
[150] R. Polikar, L. Upda, S. Upda, V. Honavar, Learn++: an incremental learning algorithm for supervised neural networks, IEEE Trans. SMC, Part C 31 (4) (2001) 497–508.
[151] R. Polikar, Ensemble based systems in decision making, IEEE Circuit Syst. Mag. 6 (3) (2006) 21–45.
[152] M. Pratama, S. Anavatti, P. Angelov, E. Lughofer, Panfis: a novel incremental learning machine, IEEE Trans. Neural Netw. Learn. Syst. 25 (1) (2014) 55–67.
[153] M. Pratama, S. Anavatti, E. Lughofer, GENEFIS: Towards an effective localist network, IEEE Trans. Fuzzy Syst. 22 (3) (2014) 547–562.

[154] M. Pratama, S. Anavatti, M. Er, E. Lughofer, Pclass: an effective classifier for streaming examples, IEEE Trans. Fuzzy Syst. 23 (2) (2015) 369–386.
[155] M. Pratama, S. Anavatti, J. Lu, Recurrent classifier based on an incremental meta-cognitive scaffolding algorithm, IEEE Trans. Fuzzy Syst. 23 (6) (2015) 2048–2066.
[156] M. Pratama, J. Lu, E. Lughofer, G. Zhang, S. Anavatti, Scaffolding type-2 classifier for incremental learning under concept drifts, Neurocomputing 191 (2016) 304–329.
[157] M. Pratama, J. Lu, E. Lughofer, G. Zhang, M. Er, Incremental learning of concept drift using evolving type-2 recurrent fuzzy neural network, IEEE Trans. Fuzzy Syst. 25 (5) (2017) 1175–1192.
[158] M. Pratama, E. Lughofer, M. Er, S. Anavatti, C. Lim, Data driven modelling based on recurrent interval-valued metacognitive scaffolding fuzzy neural network, Neurocomputing 262 (2017) 4–27.
[159] M. Pratama, W. Pedrycz, E. Lughofer, Evolving ensemble fuzzy classifier, IEEE Trans. Fuzzy Syst. 26 (5) (2018) 2552–2567.
[160] M. Pratama, E. Dimla, T. Tjahjowidodo, E. Lughofer, W. Pedrycz, Online tool condition monitoring based on parsimonious ensemble+, IEEE Trans. Cybern. (2018), doi:10.1109/TCYB.2018.2871120.
[161] R.E. Precup, T.A. Teban, A. Albu, A.I. Szedlak-Stinean, C.A. Bojan-Dragos, Experiments in incremental online identification of fuzzy models of finger dynamicsRomanian, J. Inf. Sci. Technol. 21 (4) (2018) 358–376.
[162] J.R. Quinlan, Induction of decision trees, Mach Learn 1 (1) (1986) 81–106.
[163] H.J. Rong, N. Sundararajan, G.B. Huang, P. Saratchandran, Sequential adaptive fuzzy inference system (SAFIS) for nonlinear system identification and prediction, Fuzzy Sets Syst. 157 (9) (2006) 1260–1275.
[164] H.J. Rong, N. Sundararajan, G.B. Huang, G.S. Zhao, Extended sequential adaptive fuzzy inference system for classification problems, Evolv. Syst. 2 (2) (2011) 71–82.
[165] J. Rubio, Sofmls: online self-organizing fuzzy modified least square network, IEEE Trans. Fuzzy Syst. 17 (6) (2009) 1296–1309.
[166] J. Rubio, Stability analysis for an on-line evolving neuro-fuzzy recurrent network, in: P. Angelov, D. Filev, N. Kasabov (Eds.), Evolving Intelligent Systems: Methodology and Applications, John Wiley & Sons, pp. 173–199, New York, 2010.
[167] J. Rubio, P. Angelov, J. Pacheco, Uniformly stable backpropagation algorithm to train a feedforward neural network, IEEE Trans. Neural Netw. 22 (3) (2011) 356–366.
[168] J. Rubio, A. Bouchachia, Msafis: an evolving fuzzy inference system, Soft comput. 21 (9) (2017) 2357–2366.
[169] J. Sankaranarayanan, H. Samet, A. Varshney, A fast all nearest neighbor algorithm for applications involving large point-clouds, Comput. Graph. 31 (2) (2007) 157–174.
[170] M. Sayed-Mouchaweh, E. Lughofer, Learning in Non-Stationary Environments: Methods and Applications, Springer, New York:, 2012.
[171] C. Schaffer, Overfitting avoidance as bias, Mach. Learn. 10 (2) (1993) 153–178.
[172] B. Settles, Active Learning, Morgan & Claypool Publishers, Carnegie Mellon University, 2012.
[173] A. Shaker, E. Lughofer, Self-adaptive and local strategies for a smooth treatment of drifts in data streams, Evolv. Syst. 5 (4) (2014) 239–257.
[174] A.M. Silva, W. Caminhas, A. Lemos, F. Gomide, A fast learning algorithm for evolving neofuzzy neuron, Appl. Soft Comput. 14 (B) (2014) 194–209.
[175] E. Soares, P. Costa, B. Costa, D. Leite, Ensemble of evolving data clouds and fuzzy models for weather time series prediction, Appl. Soft Comput. 64 (2017) 445–453.
[176] B.H. Soleimani, C. Lucas, B.N. Araabi, Recursive gath-geva clustering as a basis for evolving neuro-fuzzy modeling, Evolv. Syst. 1 (1) (2010) 59–71.
[177] K. Subramanian, S. Suresh, N. Sundararajan, A metacognitive neuro-fuzzy inference system (mcfis) for sequential classification problems, IEEE Trans. Fuzzy Syst. 21 (6) (2013) 1080–1095.
[178] K. Subramanian, A.K. Das, S. Sundaram, S. Ramasamy, A meta-cognitive interval type-2 fuzzy inference system and its projection based learning algorithm, Evolv. Syst. 5 (4) (2014) 219–230.
[179] S. Suthaharan, Support vector machine, in: Machine Learning Models and Algorithms for Big Data Classification, Springer, 2016, pp. 207–235.
[180] I. Škrjanc, Confidence interval of fuzzy models: an example using a waste-water treatment plant, Chemometr. Intell. Lab. Syst. 96 (2009) 182–187.
[181] I. Škrjanc, Evolving fuzzy-model-based design of experiments with supervised hierarchical clustering, IEEE Trans. Fuzzy Syst. 23 (4) (2015) 861–871.
[182] I. Škrjanc, D. Dovžan, Evolving Gustafson-Kessel possibilistic c-means clustering, Procedia Comput. Sci. 53 (2015) 191–198.
[183] I. Škrjanc, S. Blažič, Fuzzy Model-based Control - Predictive and Adaptive Approaches, Angelov, Plamen. Handbook on Computational Intelligence, World Scientific, pp. 209–240, New Jersey [etc.], 2016.
[184] I. Škrjanc, G. Andonovski, A. Ledezma, J. O. Sipele, A. Iglesias, A. Sanchis, Evolving cloud-based system for the recognition of drivers' actions, Expert Syst. Appl. (2017) 1–8.
[185] I. Škrjanc, S. Ozawa, T. Ban, D. Dovžan, Large-scale cyber attacks monitoring using evolving cauchy possibilistic clustering, Appl. Soft Comput. J. 62 (2018) 592–601.
[186] I. Škrjanc, Blažič, E. Lughofer, D. Dovžan, Inner matrix norms in evolving cauchy possibilistic clustering for classification and regression from data streams, Inf. Sci. 478 (2019) 540–563, doi:10.1016/j.ins.2018.11.040.
[187] L. Teslić, B. Hartmann, O. Nellis, I. Škrjanc, Nonlinear system identification by Gustafson-Kessel fuzzy clustering and supervised local model network learning for the drug absorption spectra process, IEEE Trans. Neural Netw., ISSN 1045–9227 22 (12) (2011) 1941–1951.
[188] Y. Tsypkin, Adaptation and learning in automatic systems, Academic Press, New York, NY, USA, 1971.
[189] Y. Tsypkin, Foundations of the Theory of Learning Systems, Academic Press, New York, NY, USA, 1973.
[190] S. Tung, C. Quek, C. Guan, Et2FIS: an evolving type-2 neural fuzzy inference system, Inf. Sci. 220 (2013) 124–148.
[191] S.G. Tzafestas, K.C. Zikidis, NeuroFAST: on-line neuro-fuzzy ART-based structure and parameter learning TSK model, IEEE Trans. Syst., Man Cybern. - Part B 31 (5) (2001) 797–802.
[192] P.E. Utgoff, Incremental induction of decision trees, Mach. Learn. 4 (2) (1989) 161–186.
[193] V. Vapnik, Statistical Learning Theory, Wiley, New York:, 1998.
[194] P. Werbos, Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences, Harvard University, 1974 Phd dissertation.
[195] D.H. Wolpert, Stacked generalization, Neural Netw. 5 (2) (1992) 241–259.
[196] S. Wu, M.J. Er, Dynamic fuzzy neural networks - a novel approach to function approximation, IEEE Trans. Syst. Man Cybern. - Part B 30 (2) (2000) 358–364.
[197] S. Wu, M.J. Er, Y. Gao, A fast approach for automatic generation of fuzzy rules by generalized dynamicc fuzzy neural networks, IEEE Trans. Fuzzy Syst. 9 (4) (2001) 578–594.
[198] R. Xiao, J. Wang, F. Zhang, An approach to incremental svm learning algorithm, in: Proc IEEE Int Conf on Tools with Artificial Intelligence, ICTAI, 2000, pp. 268–273.
[199] R.R. Yager, A model of participatory learning, IEEE Trans. Syst., Man. Cybern. 20 (5) (1990) 1229–1234.
[200] L. Zadeh, Fuzzy sets, Inf. Control 8 (3) (1965) 338–353.
[201] L. Zadeh, The concept of a linguistic variable and its application to approximate reasoning, Inf. Sci. 8 (3) (1975) 199–249.
[202] A. Zdešar, D. Dovžan, I. Škrjanc, Self-tuning of 2 DOF control based on evolving fuzzy model, Appl. Soft Comput. 19 (2014) 403–418.
[203] G.P. Zhang, Neural networks for classification: a survey, IEEE Trans. Syst., Man, Cybern., Part C (Appl. Rev.) 30 (4) (2000) 451–462.